

A Parallel Genetic Algorithm With Variable Neighborhood Search for the Vehicle Routing Problem in Forest Fire-Fighting

Xiang Li¹, Nan Chen, Hongguang Ma¹, Fapeng Nie¹, and Xianzhe Wang¹

Abstract—With the challenge of making online decisions in the face of escalating losses, the vehicle routing problem in forest fire-fighting places great emphasis on the computing efficiency of algorithms. This paper investigates the route optimization for fire-fighting vehicles and formulates a mixed integer linear programming model to minimize the total losses of overall fire spots. A parallel genetic algorithm with variable neighborhood search (PGA-VNS) is developed, which includes a parallelized evolution process of multiple sub-populations for ensuring the fast convergency to a high-quality solution, and a multi-operator variable searching process to promote the convergency to a nearly optimal solution. By comparing the computational results for various-scale instances, the PGA-VNS is demonstrated to be more effective than the baseline algorithms (i.e., Gurobi, GA, PSO, ALNS, VNS, GA-ALNS, and GA-VNS). Furthermore, the proposed model is extended to the scenario of unmanned aerial vehicles (UAVs), taking into account limited flight range and load impact constraints. The results could support the scheduling and routing for fire engines or UAVs in minimizing the losses caused by forest fires.

Index Terms—Forest fire-fighting, vehicle routing, metaheuristic algorithm, UAV.

I. INTRODUCTION

FOREST fires generally refer to large-scale uncontrolled fires that occur in forest areas, which pose significant threats to the natural environment and human society. According to the newly released Global Forest Watch analysis, the annual loss of tree cover due to fire has increased by 3 million hectares compared to 2001.¹ In terms of specific countries, Russia, Canada and the United States combined lost 7.8 million hectares of tree cover from fires in 2021; the Australia wildfires in February 2022 caused the deaths of at least 33 individuals and approximately 1 billion wildlife; the

total area of forest fires in China reached 485,000 hectares within the decade of 2010-2019.² Forest fires tend to spread over a large area, and multiple fires may occur simultaneously, requiring large amounts of rescue resources and taking a long time to implement fire-fighting operations. Due to the frequent occurrence of forest fires and their serious consequences, the development of efficient emergency rescue methods has become crucial and urgent.

Forest fires usually spread over a larger area and multiple fire spots may occur simultaneously, which requires a large amount of rescue resources and takes longer times to implement fire-fighting operations. Besides, forest fires are influenced by various factors such as terrain and the type of combustible materials, making fire-fighting operations more complex and difficult to control. The whole fire-fighting process includes collecting fire information, assessing fire levels, initiating emergency plans, executing fire suppression plans (i.e., organizing and commanding, emergency communications, vehicle dispatching and routing, logistics and medical services, and propaganda), fighting fires, reporting the results of the accident and conducting post-disaster disposal and aftercare [1]. In the above process, a reasonable dispatching and routing scheme of vehicles is imperative for minimizing the accumulated losses. To cope with forest fires, the vehicles are generally equipped with fire-fighting shells, which can be thrown to fire spots to extinguish the fires efficiently. Figure 1 shows the fire engine loaded with fire-fighting shells.³ With the continuous technological advances in recent years, Unmanned Aerial Vehicles (UAVs) have emerged as useful tools for surveillance, rescue, and prevention of forest fires without being impeded by road conditions, especially at high altitudes [2], as shown in Figure 2.⁴ Both fire-fighting vehicles and UAVs play an important role in assisting people in need in a timely manner. Among them, fire-fighting vehicles are equipped with high-powered engines, specialized chassis and suspension systems, large pumps and water storage tanks, retractable ladders and firefighting equipment. They can provide large amounts of water for fire-fighting and spray water from high places. However, the mobility of fire trucks is

Manuscript received 20 August 2023; revised 17 February 2024; accepted 24 April 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 71931001, in part by Beijing Nova Program under Grant 20230484421, in part by Beijing Social Science Foundation under Grant 22GLC059, in part by the Innovation Centre for Digital Business and Capital Development of Beijing Technology and Business University under Grant SZSK202204, and in part by the Funds for First-Class Discipline Construction under Grant XK1802-5. The Associate Editor for this article was J. Blum. (Corresponding author: Hongguang Ma.)

The authors are with the School of Economics and Management, Beijing University of Chemical Technology, Beijing 100029, China, and also with the School of Economics and Management, Chang'an University, Xi'an 710064, China (e-mail: mahg@buct.edu.cn).

Digital Object Identifier 10.1109/TITS.2024.3395930

¹<https://www.globalforestwatch.org/blog/data-and-research/trends-tree-loss-from-fires-unprecedented-detail>

²<http://www.forestry.gov.cn/sлга/3122/20210723/104902888131917.html>

³<https://www.rosenbauer.com/en/int/rosenbauer-world/news/newsletter-subscription/newsletter-2018-03/compact-simple-efficient-the-new-tlf-1500-for-zeeland-fire-brigade>

⁴<https://www.163.com/dy/article/G235J7R605149OCK.html>



Fig. 1. A fire engine with shells.



Fig. 2. A UAV with shells.

limited by road conditions and terrain, and the skill level of drivers and operators. In contrast, unlike traditional fixed-wing aircraft, UAVs are capable of utilizing a range of mobility features, such as fixed wing, rotary wing, or hybrid vertical take-off and landing (VTOL) systems [3]. These maneuvering characteristics allow UAVs to have their own unique contributions. For example, in situations where rapid response, aerial visualization, or access to hazardous areas is required, drones can provide rescue teams with new perspectives and reference information. However, the use of UAVs also presents challenges such as limited flight range and increased difficulty in maneuvering.

The dispatching and routing of vehicles in forest fire-fighting belongs to the vehicle routing problem (VRP), which aims to determine the optimal routes for vehicles, i.e., visiting orders among multiple fire spots. To improve rescue efficiency, existing studies focus on how to rationally allocate rescue resources and design optimal routes for rescuers and vehicles [4]. However, there is currently no relevant research on the relationship between loss and rescue time. The advantage of considering effective resource allocation is that it ensures the optimal distribution of limited resources such as fire-fighting equipment, personnel, and aircraft to meet the demands of different areas. This approach maximizes the effectiveness of rescue operations and reduces human and property losses. However, there are also some drawbacks to prioritizing efficient resource allocation alone. It is vital to note that response time plays a crucial role in preventing the escalation of fires and mitigating their impact. By understanding the correlation between response time and damages, researchers can determine specific time thresholds. Once these thresholds are surpassed, the efficacy of containment measures experiences a significant decline. This knowledge provides valuable information for the decision-making process and helps avoid delays that could exacerbate destruction. Furthermore, considering the relationship between response time and losses can reveal the effectiveness of different rescue strategies, potentially leading to improvements in rescue procedures. In this paper, we highlight the VRP in forest fire-fighting in which

each vehicle is assigned an identical quantity of fire-fighting shells to minimize the accumulated time-varying losses at all fire spots. To construct a universal formulation, the flow balance constraints, vehicle capacity constraints, and rescue time horizon constraints are involved. In addition, we extend the model to the scenario of using UAVs, with consideration of the flight endurance and load impact constraints.

Forest fire-fighting and rescue operations are time-sensitive such that any delay in calculating the optimal routes may lead to an escalation of losses. Therefore, there is an urgent requirement to develop feasible solutions that can assist decision makers in navigating the fire-fighting process. It is well known that VRP is NP-hard, and the solution methods include exact algorithms, classical heuristics, and metaheuristics. In general, exact algorithms are only practical for small-scale problems, while metaheuristic algorithms, such as genetic algorithms (GA) [5], [6], particle swarm optimization (PSO), variable neighborhood search (VNS) [7], [8], ant colony optimization (ACO) [9], [10], iterative local search [11] and adaptive large neighborhood search (ALNS) [12], could solve large-scale VRPs efficiently. However, in many cases, individual metaheuristics can not produce excellent results, and to address this deficiency, a general optimization framework is to combine different metaheuristics to form hybrid metaheuristics algorithms. One prevalent way is to embed single-solution-based metaheuristics (e.g., VNS, ALNS) in population-based metaheuristics (e.g., GA, PSO and ACO), thus fully utilizing the exploratory capability of population-based metaheuristics and the exploitation capability of single-solution-based metaheuristics. In literature, GA is the most widely used population-based approach combined with other methods by two forms: sequential and nested, such as the combinations of GA and simulated annealing algorithm [13], GA and branch-and-cut algorithm [14], GA and VNS [15]. Additionally, the computation time of the metaheuristics algorithm tends to increase exponentially. To address this issue for large-scale problems, parallel computing can be used to accelerate the convergency process by running multiple processors simultaneously. In a parallel computing scheme, each processor runs independently, and the best solution obtained by each processor would continuously transmit to other processors [16]. In this paper, a parallel GA with VNS (PGA-VNS) is proposed, which integrates the exploration ability of population-based algorithm based on parallel computing and the exploitation ability of single-solution-based metaheuristic algorithm, thus possessing superior performance.

In the domain of VRP in forest fire-fighting, this paper presents a triple-fold contribution as follows:

(1) A mixed-integer linear programming (MILP) model is formulated for the VRP in forest fire-fighting, which catches the characteristic of the time-varying losses at fire spots.

(2) A PGA-VNS is proposed, which integrates a sub-population parallel computing approach to improve the convergency speed of the population-based GA, and combines the single-solution-based VNS to enhance the solution quality, thus improving the practical applicability of the algorithm.

(3) The model and algorithm are extended to the scenario of UAVs, taking into account the limited flight range and load

effects. This extension supports the use of advanced technical equipment as an auxiliary rescue method to improve efficiency in forest fire-fighting.

The remainder of this paper is organized as follows. Section II reviews the related literature. Section III describes the problem and formulates a mathematical model. The PGA-VNS is designed in Section IV. Section V conducts numerical experiments to evaluate the performance of the model and algorithm. Section VI shows the extension. Section VII concludes this paper and discusses further research directions.

II. LITERATURE REVIEW

In this section, a succinct overview on two streams of literature is presented: the VRP in forest fire-fighting and the metaheuristic methods for VRP.

A. VRP in Forest Fire-Fighting

The existing studies about emergency management for forest fires center around facilities location [17], resource deployment and allocation [18], [19], and vehicle scheduling and routing [20], [21], [22]. Among these, vehicle scheduling and routing plays a crucial role in enhancing responsiveness and reducing losses.

In the past few decades, scholars have focused on the vehicle scheduling and routing issues in forest fire-fighting, which can be classified as a special type of VRP [23]. For instance, Shahparvari et al. [24] proposed an integer programming model with time windows and capacitated resources, to determine the number of required vehicles, routes and schedules for evacuees in a bushfire. Subsequently, Shahparvari et al. [25] developed a VRP model to determine the safest routes and schedules for late evacuees in bushfires under different time windows and road disruption risks. A GA-based heuristic solution method was developed to address the high complexity of VRP due to interdependencies between operations. Wu et al. [1] considered the temperature, wind force and terrain slope factors, and investigated the VRP for multi-point forest fires with the objective of minimizing the fire-fighting time and the number of dispatched engines simultaneously. They subsequently developed an exact dynamic programming algorithm and a quick greedy heuristic approach to address small-scale and large-scale instances. Fang et al. [26] addressed an emergency VRP in forest fire-fighting to design the vehicle routes among multiple fire spots with priority levels, and formulated an integer programming model with the objective of minimizing the total distance traveled by rescue teams. Tian et al. [20] established a multi-objective scheduling model to minimize the number of fire engines, the fire-fighting time, and the fire-fighting delay time.

In addition to fire engines, UAVs have become an important tool used by fire departments in recent years due to their advantages such as low cost, easy operation, and flexible movement. UAV-based forest fire-fighting research has shown increasing promise. Yuan et al. [27] presented a systematic overview of current progress in this field. Kumar and

Cohen [28] and Chen et al. [29] studied the cooperative control method of multiple UAVs engaged in extinguishing a forest fire. Saikin et al. [30] presented an approach for accurately dropping a relatively large amount of fire retardant, water or some other extinguishing agent onto a wildfire from an autonomous UAV, close to the epicenter of the fire. Other influential work includes Macrina et al. [2] and Munawar et al. [31].

Through the literature review mentioned above, it is found that few studies have concerned about the effect of time-varying loss on the vehicle routing decisions, which should not be ignored in forest fire-fighting. Regarding this, this study discretizes the continuous rescue time horizon into multiple time periods and formulates time period dependent losses at each fire spot in the model. In addition, we first present a general model for fire engines, and then extend it to the scenario of UAVs.

B. Metaheuristic Methods for VRP

Metaheuristics have been extensively utilized in the existing solution methods for VRP. For example, in Baker and Ayechew [5], a GA was implemented to address single depot VRPs with weight restrictions and distance constraints. Roozbeh et al. [12] proposed an attribute-based ALNS algorithm to address the asset protection problem of critical infrastructure during a fire incident. This problem exhibits a significant resemblance to the VRP. In this scenario, rescue operations need to be conducted within a specified time window that is influenced by the progression of the fire. To decrease carbon dioxide emissions in the VRP within urban areas, Ehmke et al. [32] enhanced an existing tabu search algorithm, whose effectiveness was validated through experiments with varying numbers of vehicles, vehicle weights, and delivery quantities.

To improve the efficiency of solving intricate problems, a prevalent strategy is to integrate different metaheuristics, which has been widely used in tackling variants of VRP. Sadati and Çatay [7] developed a hybrid VNS and tabu search approach by introducing new problem-specific neighborhood structures, which could lead to high-quality solutions within a short computation time. To address the complex routing problem for distance-constrained multi-based UAVs, Ozkan [33] proposed a metaheuristics algorithm that combines simulated annealing and local search. Furthermore, Altabeeb et al. [34] proposed a firefly algorithm (FA) coupled with two types of local search and a GA to improve solution quality and convergence rate for the capacitated VRP. Their method outperformed other existing FA variants in most of the test instances. Wu et al. [35] studied the transportation problem in emergency rescue distribution using an enhanced hybrid algorithm combining ACO and VNS with five neighborhood structures.

Recently, some studies used parallel computing methods to speed up the convergence process for metaheuristics. Cordeau and Maischberger [36] introduced a parallel iterative tabu search algorithm to solve four distinct routing problems. The heuristic method integrates tabu search with a simple perturbation mechanism which could execute parallel computations efficiently by making use of multi-core processors.

In Amirteimoori et al. [16], the parallel computing in job and carrier scheduling was investigated, in which a significant reduction in runtime was achieved by appropriately designing a parallel PSO-GA.

Furthermore, in the existing literature, both the PGA and the VNS algorithms have garnered significant attention in the field of optimization. The PGA is praised for its strong global search capability, high parallelism, and adaptability. For instance, Vincent et al. [37] utilized GA and PSO to calculate feasible and quasi-feasible paths for UAVs, yielding promising results. Zhang et al. [38] developed a parallel GA based on the Kuhn-Munkres algorithm to solve the set-covering problem and applied it to maximize the lifetime of wireless sensors. On the other hand, the VNS algorithm functions as a local search algorithm that rapidly converges to optimal solutions by varying the neighborhood structure of the problem solution. This algorithm has been widely applied due to its robustness and ease of implementation [7], [42]. Despite the commendable achievements of both PGA and VNS in their respective research directions, the integration of these two algorithms has received limited attention. The fusion of PGA and VNS enables the utilization of PGA's global search capability and adaptability, as well as VNS's fast convergence and robustness. This combination not only expands the solution space for optimization but also provides new perspectives for solving complex optimization problems. Motivated by the previous literature, this study proposes a PGA-VNS by decomposing the complete population of GA into sub-populations for parallel evolution with VNS injected as a local search. A detailed explanation will be elaborated in Section IV.

III. PROBLEM DESCRIPTION AND MODEL FORMULATION

The VRP in forest fire-fighting is defined on a directed graph $\mathbb{G}(\mathbb{V}, \mathbb{A})$. Here $\mathbb{V} = \{0\} \cup \{N+1\} \cup \mathbb{V}'$ is the set of vertices, where $\mathbb{V}' = \{1, 2, \dots, N\}$ denotes the set of fire spots, vertex 0 represents the depot, and vertex $N+1$ represents the virtual depot. $\mathbb{A} = \{(i, j) | i, j \in \mathbb{V}, i \neq j\}$ is the collection of arcs connecting vertices, and t_{ij} is the travel time associated to arc $(i, j) \in \mathbb{A}$. The travel time on each arc is assumed to be known, which can be calculated based on the distance (estimated by satellite) and vehicle speed. Note that the travel time from each fire spot to the virtual depot is taken to be zero, that is, $t_{i(N+1)} = 0, \forall i \in \mathbb{V}'$. Suppose that there are K homogeneous vehicles (fire engines) located at the depot, and each vehicle is loaded with fire-fighting shells that have powerful fire-fighting ability, and the maximum capacity of each vehicle is denoted by Q . The overall rescue time horizon is defined as $[T_s, T_e]$. When a fire alarm occurs, to ensure a prompt and efficient response, the vehicles synchronously depart from the depot at time T_s . Each vehicle visits a series of assigned fire spots where fire-fighting activities take place and finally returns to the virtual depot after completing all fire-fighting tasks. The operation time for each fire-fighting shell is h , including installing fire-fighting shells on the transmitter, adjusting the distance and angle between the transmitter and the fire spot, launching fire-fighting shells, etc.

The fire spots are mutually independent, that is, the fire situation at one spot is not affected by others. For each spot,

the spread rate R of fire is considered to be nonlinear taking into account natural factors, such as fuel types, wind force and terrain slope as shown in [1],

$$R = R_0 \lambda_s \lambda_\phi \lambda_\omega = R_0 \lambda_s \lambda_\phi e^{0.1783 v_\omega}, \quad (1)$$

where $R_0 = \alpha T_{em} + \theta \omega + \gamma$ denotes the initial spread speed, T_{em} and ω represent the temperature and wind force, respectively; α , θ and γ are the factors related with the terrain which are determined by the actual location of forest fires; λ_s , λ_ϕ , and λ_ω are the correction factors of fuel types, terrain slope, and wind force, respectively, and v_ω represents the wind speed. During the overall rescue time horizon $[T_s, T_e]$, the loss function at different times t is simulated according to the function

$$f_i(t) = \eta_i R_i^{\sqrt{t}}, \quad i = 1, 2, \dots, N, \quad (2)$$

where η_i represents the observed initial fire situation at spot i , and R_i represents the deduced spread rate of fire development. Here, $f_i(t)$ is a continuous function with non-linear features, and by approximating the non-linear terms with their linear counterparts, the equation can be transformed into a linear form, preserving its tractability and facilitating subsequent solution process. We split the time horizon $[T_s, T_e]$ into T periods with the same interval of time segment Δ , where $\delta = 1, 2, \dots, T$. Furthermore, let $T+1$ denotes the period beyond the prescribed rescue time horizon. In other words, the loss function of fire spot i is modified to $f_{i\delta}$ as shown in Eq. (3). Notably, the value of $f_{i\delta}$ is taken as the left endpoint of $f_i(t)$ within the corresponding δ period. Therefore, the time interval should not be too wide to avoid significant errors.

$$f_i(t) = \begin{cases} f_{i1}, & \text{if } T_s \leq t < T_s + \Delta \\ f_{i2}, & \text{if } T_s + \Delta \leq t < T_s + 2\Delta \\ \vdots & \\ f_{i\delta}, & \text{if } T_s + (\delta - 1)\Delta \leq t < T_s + \delta\Delta \\ \vdots & \\ f_{iT}, & \text{if } T_s + (T - 1)\Delta \leq t < T_e \\ f_{i(T+1)}, & \text{if } T_e \leq t \end{cases} \quad (3)$$

This paper aims at determining the optimal visiting routes for vehicles to minimize the total losses of overall fire spots, subject to variable loss values influenced by multiple factors. Here we assume that all available vehicles are scheduled during forest fire-fighting to reduce the loss, such that the number of vehicles is handled as a given parameter instead of a decision variable.

Figure 3 shows a forest fire-fighting graph which includes one depot, six fire spots, and one virtual depot. Considering a rescue time horizon of 240 minutes and the duration of $\Delta = 40$ minutes for each period, then a total of 7 time periods are generated, where $\delta = 1, 2, \dots, 6$ correspond to the rescue period, and $\delta = 7$ corresponds to the period beyond the rescue time horizon. There are 3 fire engines with a capacity of 2 shells. In this small-scale example, the power of the fire-fighting shells is sufficient enough that each fire spot requires just one shell. For each arc, its weight denotes the travel time (in minutes) along the arc. The operation time

TABLE I
THE TIME-VARYING LOSSES AT FIRE SPOTS

Spot	Loss						
	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$	$t=6$	$t=7$
1	66.67	212.62	517.70	1096.21	2122.97	3858.82	6684.99
2	24.45	59.09	116.33	205.91	340.52	536.61	815.25
3	39.59	109.33	238.37	459.86	820.44	1384.69	2240.69
4	41.60	116.47	256.61	499.46	898.08	1526.46	2486.17
5	63.50	199.78	481.45	1010.60	1942.19	3505.82	6034.79
6	2.53	3.28	3.99	4.71	5.46	6.23	7.04

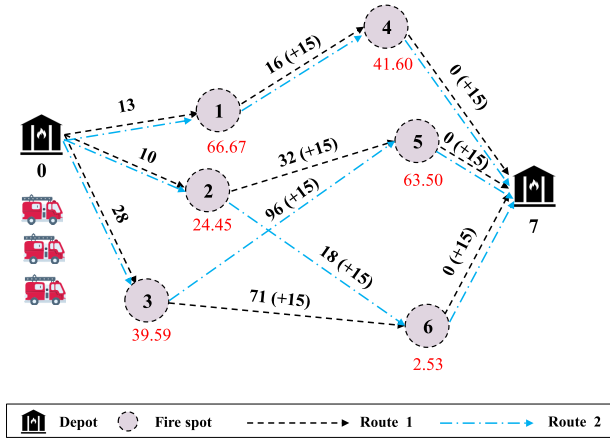


Fig. 3. Illustration of routing solutions.

TABLE II
COMPARISON OF ROUTING SOLUTIONS

	Vehicle	Route	Loss
Solution 1	1	Depot 0 → Spot 1 → Spot 4 → Depot 7	521.4
	2	Depot 0 → Spot 2 → Spot 5 → Depot 7	
	3	Depot 0 → Spot 3 → Spot 6 → Depot 7	
Solution 2	1	Depot 0 → Spot 1 → Spot 4 → Depot 7	1330.8
	2	Depot 0 → Spot 2 → Spot 6 → Depot 7	
	3	Depot 0 → Spot 3 → Spot 5 → Depot 7	

at fire spots is $h = 15$ minutes. For this problem, two feasible solutions are presented in Table II. We can see that in the first solution, vehicle 1 visits spot 1 and spot 4, vehicle 2 visits spot 2 and spot 5, and vehicle 3 visits spot 3 and spot 6, resulting in a loss of 521.4. In the second solution, vehicle 1 visits spot 1 and spot 4, vehicle 2 visits spot 2 and spot 6, and vehicle 3 visits spot 3 and spot 5, resulting in a loss of 1330.8. Obviously, the first solution outperforms the second solution in terms of accumulated loss.

In the following, we formulate a novel MILP model for the VRP in forest fire-fighting. For a convenient understanding, all the notations used are summarized in Table III. To minimize the total losses of overall fire spots, the objective function is defined as follows

$$C = \sum_{i=1}^N \left(\sum_{k=1}^K \sum_{\delta=1}^T f_{i\delta} z_{i\delta}^k + f_{i(T+1)} \left(1 - \sum_{k=1}^K y_i^k \right) \right). \quad (4)$$

The first term represents the losses at fire spots that are visited by vehicles, and the second term represents the losses at fire spots that are not visited by any vehicles.

TABLE III
NOTATIONS

Indices	
k	Index of vehicles, $k = 1, 2, \dots, K$
i, j	Indices of vertices, $i, j = 0, 1, 2, \dots, N+1$, where N is total number of fire spots, and 0 and $N+1$ represent depots
δ	Index of time intervals, $\delta = 1, 2, \dots, T+1$
Parameters	
t_{ij}	Travel time from vertex i to vertex j
$f_{i\delta}$	The loss value of fire spot i if vehicle k arrives at fire spot i in time interval δ , $i = 1, 2, \dots, N$, $\delta = 1, 2, \dots, T+1$
$[T_s, T_e]$	Overall rescue time horizon
Q	The maximum number of fire-fighting shells loaded by each vehicle
h	The single operation time at fire spots
Δ	The interval of time periods
B	The power of each fire-fighting shell
Decision variables	
x_{ij}^k	Taking value 1 if vehicle k travels from fire spot i to fire spot j , 0 otherwise, $i = 0, 1, \dots, N$, $j = 1, 2, \dots, N+1$
y_i^k	Taking value 1 if vehicle k visits fire spot i , 0 otherwise, $i = 1, 2, \dots, N$
Intermediate variables	
AT_i^k	The time when vehicle k arrives at fire spot i , $i = 0, 1, 2, \dots, N+1$
$z_{i\delta}^k$	Taking value 1 if vehicle k arrives at fire spot i in time period δ , 0 otherwise, $i = 0, 1, 2, \dots, N+1$, $\delta = 1, 2, \dots, T+1$

As a VRP, the flow balance constraint is required. Specifically, each vehicle should start its travel at depot 0 and end its travel at virtual depot $N+1$, which can be described as

$$\sum_{j=1}^N x_{0j}^k = 1, \quad k = 1, 2, \dots, K, \quad (5)$$

$$\sum_{j=1}^N x_{j(N+1)}^k = 1, \quad k = 1, 2, \dots, K. \quad (6)$$

In addition, each vehicle k should arrive at and leave from the same fire spot, that is,

$$\sum_{j=0}^N x_{ji}^k = \sum_{j=1}^{N+1} x_{ij}^k, \quad k = 1, 2, \dots, K, i = 1, 2, \dots, N. \quad (7)$$

Since the number of available vehicles is limited, a fire spot can be visited by vehicles at most once, and some fire spots may not even be visited by any vehicles, so we have

$$\sum_{k=1}^K y_i^k \leq 1, \quad i = 1, 2, \dots, N. \quad (8)$$

Due to the substantial variations in fire intensity and spread rate at each fire spot, the required fire-fighting resources differ over time as well. $\left\lceil \frac{f_{i\delta} z_{i\delta}^k}{B} \right\rceil$ indicates the demand of fire-fighting shells if vehicle k visits fire spot i during δ , where B represents the power of each fire-fighting shell. For any vehicle, the number of fire-fighting shells loaded is constrained

by the vehicle capacity, that is

$$\sum_{i=1}^N \sum_{\delta=1}^T \left\lceil \frac{f_{i\delta} z_{i\delta}^k}{B} \right\rceil \leq Q, \quad k = 1, 2, \dots, K. \quad (9)$$

For each vehicle, the time departing from the depot should be equal to the initial rescue time, that is

$$AT_0^k = T_s, \quad k = 1, 2, \dots, K. \quad (10)$$

For vehicle k , the departure time and the arrival time at fire spots should satisfy the following constraints

$$\begin{cases} AT_j^k \leq AT_i^k + h \left\lceil \frac{f_{i\delta} z_{i\delta}^k}{B} \right\rceil + t_{ij} + M(1 - x_{ij}^k) \\ AT_j^k \geq AT_i^k + h \left\lfloor \frac{f_{i\delta} z_{i\delta}^k}{B} \right\rfloor + t_{ij} - M(1 - x_{ij}^k), \end{cases} \quad (11)$$

where $k = 1, 2, \dots, K, i = 0, 1, 2, \dots, N, j = 1, 2, \dots, N + 1, \delta = 1, 2, \dots, T$, which formulate the relationship among arrival time, operation time and travel time along arcs.

To align with the linearization of loss values, we introduce Eq. (12), which are used to define the arrival time periods at fire spots, that is

$$\begin{cases} (T_s + (\delta - 1)\Delta) - AT_i^k \leq M \times (1 - z_{i\delta}^k) \\ AT_i^k - (T_s + \delta\Delta) < M \times (1 - z_{i\delta}^k), \end{cases} \quad (12)$$

where $\delta = 1, 2, \dots, T, k = 1, 2, \dots, K, i = 1, 2, \dots, N$.

Constraints (13) denote that if fire spot i is visited, then its arrival time should be at one and only one time period, that is,

$$\sum_{\delta=1}^T z_{i\delta}^k = y_i^k, \quad k = 1, 2, \dots, K, i = 1, 2, \dots, N. \quad (13)$$

Constraints (14) ensure that if there is a vehicle departing from fire spot i , then fire spot i is definitely visited, that is,

$$\sum_{j=1}^{N+1} x_{ij}^k = y_i^k, \quad k = 1, 2, \dots, K, i = 1, 2, \dots, N. \quad (14)$$

Constraints (15)-(17) define the domain of variables.

$$AT_i^k \geq 0, \quad k = 1, 2, \dots, K, i = 0, 1, 2, \dots, N + 1, \quad (15)$$

$$x_{ij}^k \in \{0, 1\}, \quad k = 1, 2, \dots, K, i = 0, 1, 2, \dots, N, \quad (16)$$

$$y_i^k \in \{0, 1\}, \quad k = 1, 2, \dots, K, i = 1, 2, \dots, N, \quad (17)$$

According to the above objective function and constraints, a mathematical model $P1$ for the VRP in forest fire-fighting is formulated as follows:

$$\begin{cases} \min & C \\ \text{s.t.} & \text{Constraints (5)-(17)}. \end{cases} \quad (18)$$

The proposed $P1$ is essentially an MILP model, and the common branch and bound method and commercial solvers, such as Gurobi and CPLEX, are capable of solving such MILP. However, due to the NP-hard characteristic of VRP, the solving time would increase exponentially with the problem's

scale, thereby leading to the failure of solving relatively large-scale problems in an acceptable time (As shown in the experiments, Gurobi could not solve the instance with 15 fire spots and 3 vehicles with a capacity of 3 shells in one hour). Metaheuristic algorithms have been widely used to solve VRPs for the advantage of achieving high-quality solutions in a short time. Regarding the requirement for high timeliness in practice, we devote to developing an efficient metaheuristic algorithm, PGA-VNS, for the VRP in forest fire-fighting in the following.

IV. PGA-VNS HYBRID METAHEURISTIC ALGORITHM

In this section, we discuss the proposed PGA-VNS hybrid metaheuristic algorithm specifically and start with the spirit of algorithm design. GA has the advantages of fewer adjustment parameters, easy implementation, and scalability to solve various VRPs [41], which inspires us to choose GA as the core of our metaheuristic algorithm. Moreover, as a kind of population-based search algorithm, GA is naturally parallelizable, which leads us to extend GA to parallel GA using population-based parallel computing with multi-core CPUs to improve computational efficiency. However, since GA suffers from disadvantages such as premature convergence and poor local search capability, we embed the VNS algorithm to search sub-optimal solutions based on the obtained high-quality solutions of parallel GA. The VNS algorithm is a metaheuristic algorithm for solving optimization problems that could augment the searchability by systematically changing the set of neighborhood structures, thus continuously improving the current local optimal solution [42].

Figure 4 depicts the flow-chart of PGA-VNS specifically. The algorithm starts by setting the parameters such as population size, crossover probability, mutation probability, and the number of neighborhood operators. Next, we execute parallel GA as the first part of hybrid metaheuristic algorithm to search a population of high-quality solutions. The parallel GA procedure includes three main steps: the sub-population initialization, the parallel sub-iteration, and the information sharing among sub-populations. Once the parameters are inputted, multiple initial sub-populations are first generated in parallel, where each sub-population characterizes a set of routes. Then, each sub-population conducts the selection, crossover and mutation operations parallelly during their corresponding sub-iterations. Finally, all sub-populations share their information to acquire a set of higher-quality solutions. Here we take the maximum iteration number as the stopping criterion in parallel GA. During the execution of the parallel GA procedure, there are five main tasks worth highlighting: representation of the initial sub-population, selection mechanism, crossover operation, mutation operation and parallel computation, which is described in Subsection IV-A.

Based on the high-quality populations obtained from the parallel GA, we select a best-found solution X and turn to the VNS procedure for searching a sub-optimal solution. Sharing the same spirit as Hansen et al. [42], we design two sets of neighborhood structures, denoted by $N_k(X)$ and $N_l(X)$ respectively, where $N_k(X)$ is the set of neighborhood structure for the shaking procedure and $N_l(X)$ is the set of

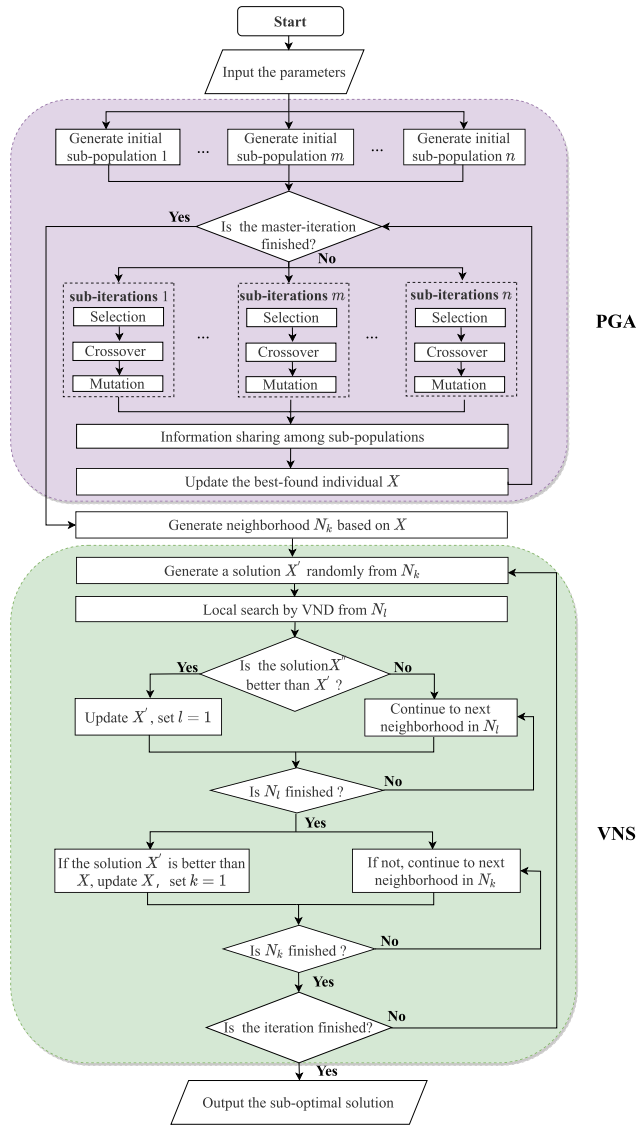


Fig. 4. Flowchart of PGA-VNS.

neighborhood structure for the variable neighborhood descent (VND) procedure. We select a neighborhood structure from $N_k(X)$ and perform a shaking process on X to generate a new solution X' , which undergoes a local search in $N_l(X')$. Finally, the shaking and VND procedure iterate until all neighborhood structures have been completed. A more detailed VNS procedure is described in Subsection IV-B. In general, metaheuristic algorithms typically terminate when they reach a predetermined number of iterations, observe a minimal improvement in solution quality over several iterations, or exhaust allocated computational resources such as time or memory. In our study, both parallel GA and VNS algorithms terminate after a given number of generations of the above steps similar to [38], [42], and [43], represented by $Iter_GA$ and $Iter_VNS$, respectively.

A. Parallel Genetic Algorithm

The algorithm framework of parallel GA has been exhibited, nevertheless, some important tasks have not been specified.

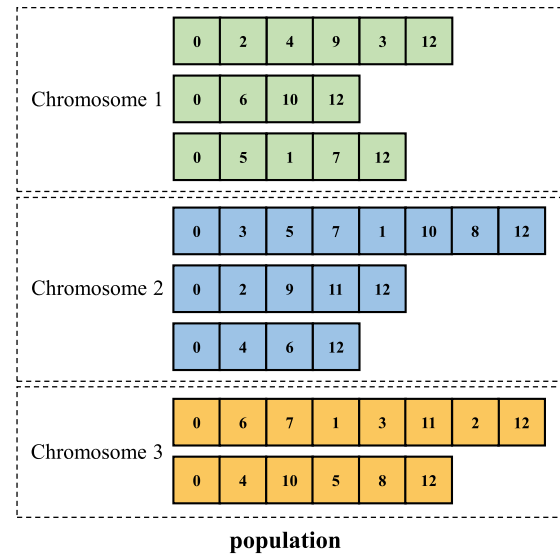


Fig. 5. Illustration of chromosome and population.

Therefore, we describe the relevant tasks of parallel GA in more detail in the following.

1) *Representation of the Initial Sub-Population*: The first challenging task execution is the design of a chromosome suitable for the VRP in forest fire-fighting. Inspired by the work of Ma et al. [43], the chromosome is represented by a multi-row matrix structure, where each row describes a vehicle route with variable length and the number of vehicles is captured by counting the number of rows. To easily visualize the chromosome structure, a sketch of the chromosome is shown in Figure 5, which illustrates a population consisting of three chromosomes. Here 0 represents the origin depot and 12 represents the destination depot, and all chromosomes have the same origin and destination depot. Taking chromosome 1 for example, it consists of 3 rows (i.e., 3 vehicles for implementing the fire-fighting tasks) where the first row $r = [0, 2, 4, 9, 3, 12]$ means that a vehicle starts at depot 0, visits fire spots 2, 4, 9, 3 sequentially, and returns to depot 12.

In the process of initializing the population, we start from generating a chromosome. For each vehicle $1 \leq k \leq K$, a random number r is generated from the set of $\{1, 2, \dots, Q\}$, and r fire spots are randomly selected from the candidate set to generate a route. Assign this route, the values of other variables z , y , and AT are calculated separately based on equations (12), (14), and (11). Then we proceed to check the cumulative demand load of each point sequentially according to equation (9), ensuring that the capacity constraints are not violated. Finally, a feasible chromosome X is generated by adding the origin and destination depot to each route. After obtaining X and the corresponding intermediate variables, the objective function can be calculated subsequently. Furthermore, repeat the chromosome generation process sub_pop_size times to obtain an initial sub-population. Finally, repeat the above process to generate n sub-populations with pop_size individuals in total.

2) *Selection Mechanism Based on Fitness Function*: The fitness function is used to evaluate the quality of a chromosome within the search space. Chromosomes are evaluated based on their corresponding fitness values to determine which one

will be inherited to the next generation by selection operation. Chromosomes with higher fitness values are more likely to be selected for reproduction. Generally speaking, the design of the fitness function may have an impact on both solution quality and computing efficiency. In this study, we first rank the chromosomes in ascending order based on their objective values, that is

$$C(X_i) \leq C(X_j), \quad \forall i \leq j, \quad i, j = 1, 2, \dots, \text{sub_pop_size}, \quad (19)$$

where $C(X_i)$ is the objective value corresponding to chromosome X_i formulated as formula (4). Then the fitness value of chromosome X_i is defined as

$$\text{Fitness}(X_i) = \alpha(1 - \alpha)^{i-1}, \quad i = 1, 2, \dots, \text{sub_pop_size}, \quad (20)$$

where parameter α is a constant between 0 and 1 [43].

The selection mechanism allows individuals with higher fitness values to be inherited while maintaining the diversity of the population. The commonly used selection mechanisms include elite selection, roulette wheel selection, tournament selection, and so on. Since the randomness of the roulette wheel strategy may result in losing the optimal chromosome, we adopt the roulette wheel selection with the elite retention strategy to determine the parents. The elite retention strategy aims to preserve the best chromosome in the population and pass it to the next generation directly. Meanwhile, in the roulette wheel selection strategy, the probability of each chromosome being selected is proportional to its fitness. Specifically, based on the fitness of each chromosome, the probability $P(X_i)$ that each chromosome can be inherited to the next generation is calculated by

$$P(X_i) = \frac{\text{Fitness}(X_i)}{\sum_{i=1}^{\text{sub_pop_size}} \text{Fitness}(X_i)}, \quad i = 1, 2, \dots, \text{sub_pop_size}, \quad (21)$$

where sub_pop_size is the size of sub-population, $\text{Fitness}(X_i)$ is the fitness value of the individual X_i and has been given in equation (20). Then the cumulative probability $q(X_i)$ of each chromosome X_i is obtained consequently according to

$$q(X_i) = \sum_{j=1}^i P(X_j), \quad i = 1, 2, \dots, \text{sub_pop_size}. \quad (22)$$

A random number r is generated from the interval $[0, 1]$, and if the cumulative probability $r \leq q(X_i)$, chromosome X_i is selected, otherwise, an individual X_m is selected according to $q(X_{m-1}) < r \leq q(X_m)$. This process is repeated for $\text{sub_pop_size} - 1$ times to generate a new sub-population.

3) *Crossover Operation*: Crossover operation increases the diversity of the population and promotes the search for global optimal solution. During the crossover operation, the sub-population is divided into $\text{sub_pop_size}/2$ pairs and each pair of parents is represented as X_1 and X_2 . A crossover

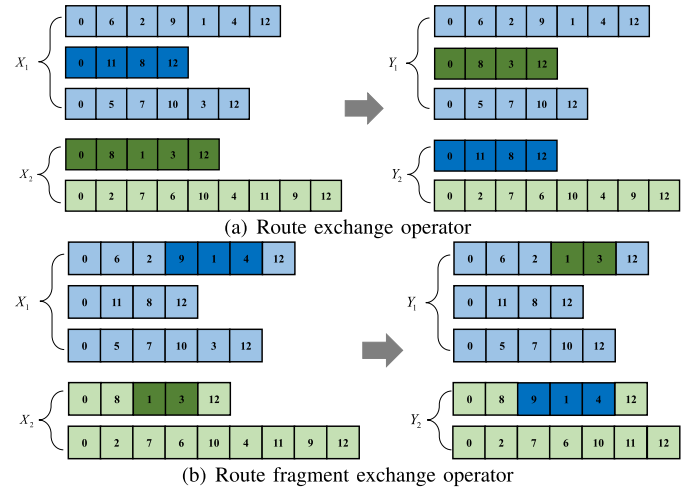


Fig. 6. Illustration of genetic crossover operators.

probability p_c is defined before the process, followed by a generation of a random number r from the interval $[0,1]$. The crossover operation is implemented if $r \leq p_c$. It should be emphasized that the application of crossover operators does not guarantee improved or similar outcomes in all cases. The evaluation criteria of the offspring's suitability is based on the corresponding fitness values. If the offspring fails to meet the criteria for enhancing or preserving the quality of the solution, it is deemed unacceptable, and the respective parent chromosome remains unaltered.

We adopt two crossover operations (i.e., route exchange operator and route fragment exchange operator), which achieve the same objective of exchanging genetic information between two parents to produce offspring, yet differ in the way of operation. For the route exchange operator, we randomly select one route from each parent and exchange the selected two routes to form offspring, as shown in Figure 6(a). For the route segment crossover operation, we randomly select a position on the chromosome and choose random segments from the positions onward in parent chromosomes X_1 and X_2 . Then, we exchange the selected segments to generate two offspring (chromosomes Y_1 and Y_2), as shown in Figure 6(b). After the crossover operation, we make feasible modifications for the offspring, which mainly focus on two aspects: (1) duplicated genes in the chromosomes after crossing should be deleted, (2) for each row of the chromosome, calculate the number of shells dropped at each point in the sequence. If the accumulated payload exceeds the vehicle capacity Q , it indicates a violation of the vehicle capacity constraint, and the excess should be deleted.

4) *Mutation Operation*: Mutation operation refers to the introduction of a random perturbation in genetic information (e.g., chromosomes or genes) to increase the diversity of a population. Through mutation operation, the algorithm can more effectively avoid premature convergence to local optimal solutions. Similar to the crossover operation, given a relatively small mutation probability p_m , a number r from the interval $[0,1]$ is randomly selected as well and the mutation operator is executed if $r \leq p_m$.

Two mutation operators (i.e., gene exchange operator and route realign operator) are proposed. For the gene exchange

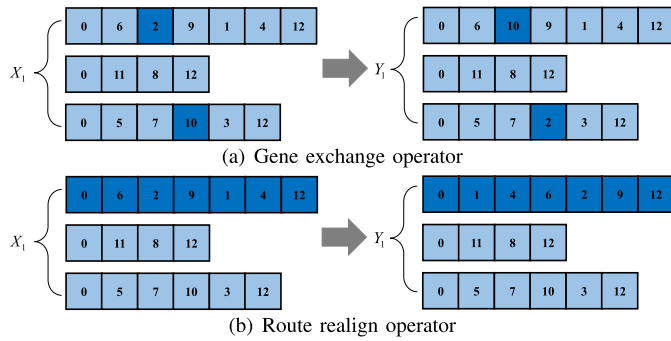


Fig. 7. Illustration of genetic mutation operators.

operator, we randomly select two routes from the parent chromosome and two positions n_1 and n_2 on them respectively. The genes at these two positions are exchanged to form the offspring chromosome Y , as illustrated in Figure 7(a). For the route realign operator, an arbitrary route is randomly selected from the parent chromosome X , then the genes except for the start and end positions are realigned and reinserted into X to generate the offspring chromosome Y , as shown in Figure 7(b).

5) *Parallel Computation*: Parallel computation can speed up solving the problem by utilizing the multi-core CPU technology to complete multiple sub-tasks simultaneously and independently. To comply with the requirements of parallel computation, sub-populations evolve independently so that the available computing resources can be fully utilized to reduce the solution time. From the perspective of algorithm implementation, the parallel GA is very convenient to achieve in multi-core machines provided by MATLAB. The n sub-populations are generated based on the number of CPU cores available, and the selection, crossover, and mutation operations are performed among each sub-population independently, where *Parfor* completes parallel computation by splitting each computational module in the independent loop and assigning it to different CPU cores. After each sub-population performs evolutionary operations, the information shared among sub-populations is completed by replacing the worst individuals in each sub-population with the best individuals from each sub-population and synchronizing the information of the best individuals to all sub-populations.

Following performing selection, crossover, and mutation operations in parallel, a new sub-population is generated, and parallel GA will terminate after a given number of generations of the above steps. The pseudocode of the parallel GA is covered in Algorithm 1.

B. Variable Neighborhood Search

VNS is a metaheuristic algorithm based on local search which has been effectively utilized for the VRP to enhance the quality of solutions [44]. Essentially, the fundamental concept behind VNS is that a local optimal solution in one neighborhood structure may not be the optimal solution in another structure, therefore, the globally optimal solution may be obtained by combining the local optimal solutions from all possible neighborhood structures. Unlike other metaheuristics based on local search, VNS obtains the local optimal solution by implementing the search operations that are limited to

Algorithm 1 Parallel Genetic Algorithm

Input: Travel time between fire spots T , Cumulative time-varying losses of all fire spots F , Crossover operator probability p_c , Mutation operator probability p_m , Population size pop_size , sub-population number n and Generation number G .

Output: The best-found chromosome X and the objective value $Fitness^*$.

```

1 Iteration = 1
2 Initialize  $n$  sub-populations consisting of  $pop\_size/n$ 
  chromosomes and get the current optimal chromosome  $S$ 
3 while Iteration ≤  $G$  do
4   parfor  $i = 1 : 1 : n$ 
5      $Fitness$  = fitness value of all chromosomes.
6      $Fitness^* = \min(Fitness)$ .
7     Execute the selection process to breed a new generation
      according to Section IV-A2.
8     for  $k = 1 : 2 : pop\_size/n$  do
9       if  $random \leq P_c$  then
10        Execute crossover operators for chromosome( $k$ )
          and get chromosomes $_1(k)$  according to
          Section IV-A3
11      end
12       $Fitness_1$  = fitness value of all chromosomes $_1$ .
13      if  $Fitness_1(k) < Fitness(k)$  then
14        chromosomes( $k$ ) = chromosomes $_1(k)$ 
15         $Fitness(k) = Fitness_1(k)$ 
16      end
17    end
18    for  $k = 1 : 1 : pop\_size/n$  do
19      Execute mutation operator for chromosomes( $k$ ) and
        get chromosomes $_2(k)$  according to Section IV-A4
20       $Fitness_2$  = fitness value of all chromosomes $_2$ 
21      if  $Fitness_2(k) < Fitness(k)$  then
22        chromosomes( $k$ ) = chromosomes $_2(k)$ 
23         $Fitness(k) = Fitness_2(k)$ 
24      end
25    end
26    Share information among  $n$  sub-populations and update
       $X$ 
27  end parfor
28  Iteration = Iteration+1
29 end

```

the neighborhoods of the solution, while the neighborhoods are obtained by systematically changing the nodes of the solution [42]. In this study, we present six neighborhood structures utilized to address our problem.

(1) The route replacement operator. An arbitrary route is randomly selected from the parent chromosome and replaced by a new route that is randomly generated. Through feasible modifications, offspring chromosome Y is generated as depicted in Figure 8(a).

(2) The route insertion operator. if the number of rows in parent chromosome X is smaller than the total number of vehicles K , a new route is randomly regenerated and inserted into the chromosome, otherwise, no changes are made. Offspring chromosome Y is generated followed by feasible modifications, as illustrated in Figure 8(b).

(3) The gene replacement operator. k routes are randomly selected from parent chromosome X . For each route, a gene n_1 located at the index that randomly selected is replaced with another randomly generated gene n_2 that satisfied $n_2 \neq$

n_1 , thus a new offspring chromosome Y generated, see Figure 8(c).

(4) The gene insertion operator. After searching for some fire spots that do not exist in the parent chromosome X , one of these spots is randomly chosen as a new gene to be inserted into chromosome X , resulting in the generation of chromosome Y , as depicted in Figure 8(d).

(5) The gene relocation operator. k routes are randomly selected from the parent chromosome X . Then, two indices n_1 and n_2 are selected from route j ($j \in [0, K]$), which are satisfied $n_2 \neq n_1$ and $n_1, n_2 \in [0, N_j]$ where N_j is the length of route j . Next, the gene located at n_1 is relocated behind position n_2 , and the segment between n_1 and n_2 is moved forward to generate a new route. Repeat the process k times and an offspring chromosome Y is generated, as shown in Figure 8(e).

(6) The greedy deletion operator. One route is randomly selected from the parent chromosome X with the genes except for the start and end nodes being deleted in sequence. The respective objective values of those chromosomes are calculated and recorded, then the gene with the minimum objective value is deleted to generate offspring chromosome Y , see Figure 8(f).

In this paper, the VNS algorithm is composed of two parts (i.e., shaking and variable neighborhood descent search (VND) process) with the same neighborhood sets. First, we shuffle the order of the neighborhoods N_k for $k = 1, 2, \dots, S$ and perform the operator on the current solution X sequentially to generate a new solution X' . Next, we conduct a VND search by selecting a neighborhood structure from N_l based on X' , performing a local search within this neighborhood structure to obtain a new local optimal solution X'' . If X'' is better than the current incumbent solution, that is $C(X'') \leq C(X')$, then the current incumbent solution is updated to X'' and the algorithm jumps back to the first neighborhood in N_l to start again, otherwise, move directly to the next neighborhood structure for further searching. This process continues until all neighborhood structures in N_l have been searched, and a new shaking process begins. VNS is distinctive in the use of a variable-neighborhood mechanism that helps it avoid getting trapped in local optimum. We take the maximum iteration number as the stopping criterion, and the pseudocode of the VNS is shown in Algorithm 2.

V. COMPUTATIONAL EXPERIMENTS

This section conducts numerical experiments to evaluate the effectiveness of the proposed model and PGA-VNS. All experiments are executed on a personal computer equipped with an Intel Core i5-4460 processor running at 3.3 GHz with 8 GB memory. The PGA-VNS is encoded by MATLAB. We rely on the Parallel Computing Toolbox in MATLAB to implement the parallel evolution of 4 sub-populations corresponding to CPU cores.

To accommodate test cases of varying sizes and ensure the robustness of the results, this study employed an instance generator to randomly construct test instance data, and the algorithm for generating these instances is designed to vary

Algorithm 2 Variable Neighborhood Search

Input: Travel time between fire spots T , Time-varying losses of fire spots F , Number of operators S , Iteration number M and Best-found solution X generated from parallel GA.

Output: The best-found solution and objective value.

```

1 Select the set of neighborhood structures  $N_k(X)$  for
   $k = 1, 2, \dots, S$ 
2  $Obj =$  objective value of  $X$ 
3  $k = 1, l = 1$ 
4 for  $m = 1 : 1 : M$  do
5   while  $k \leq S$  do
6     Generate a solution  $X'$  at random from the  $k$ th
       neighborhood of  $X(X' \in N_k(X))$ 
7      $Obj_1 =$  objective value of  $X'$ 
8     while  $l \leq S$  do
9       Find the local optimum  $X''$  in  $N_l(X')$ 
10       $Obj_2 =$  objective value of  $X''$ 
11      if  $Obj_2 < Obj_1$  then
12         $l = 1$ 
13         $X' = X'', Obj_1 = Obj_2$ 
14      else
15         $l = l + 1$ 
16      end
17    end
18    if  $Obj_1 < Obj$  then
19       $k = 1$ 
20       $X = X', Obj = Obj_1$ 
21    else
22       $k = k + 1$ 
23    end
24  end
25 end

```

key parameters, such as the locations of fire spots, the conditions affecting their spread, and the initial speed of spread. The process begins by randomly establishing the distances between fire spots. Subsequently, the expected loss values for each fire spot in different periods are calculated based on formulas (1), (2), and (3) presented in Section III, and the parameter η_i is set to $\sqrt{R_i}$. There are 24 small-scale instances (detailed in Section V-B) and 16 large-scale instances (detailed in Section V-C). Each instance is named the following format 'ax_y_z', where x is the number of fire spots, y is the number of available vehicles and z is the capacity of each vehicle. For example, instance a8_2_3 consists of 8 spots and 2 vehicles with a capacity of 3 shells. The overall rescue time horizon begins at $T_s = 0$ min and ends at $T_e = 240$ min, which is divided into 6 time periods with a duration of $\delta = 40$ min for each period. The operation time at each spot is $h = 15$ min and the parameter B is set to 1000.

A. Parameter Tuning of PGA-VNS

The crossover probability p_c , the mutation probability p_m , the population size pop_size and the maximum number of iterations G would significantly affect the performance of GA. The maximum number of iterations G is set to be 200 for the PGA-VNS [45]. First, we use the grid search method to calibrate the values of p_c and p_m based on the instance 'a15_3_4'. We enumerate the value of p_c at [0.5, 0.6, 0.7, 0.8, 0.9] and

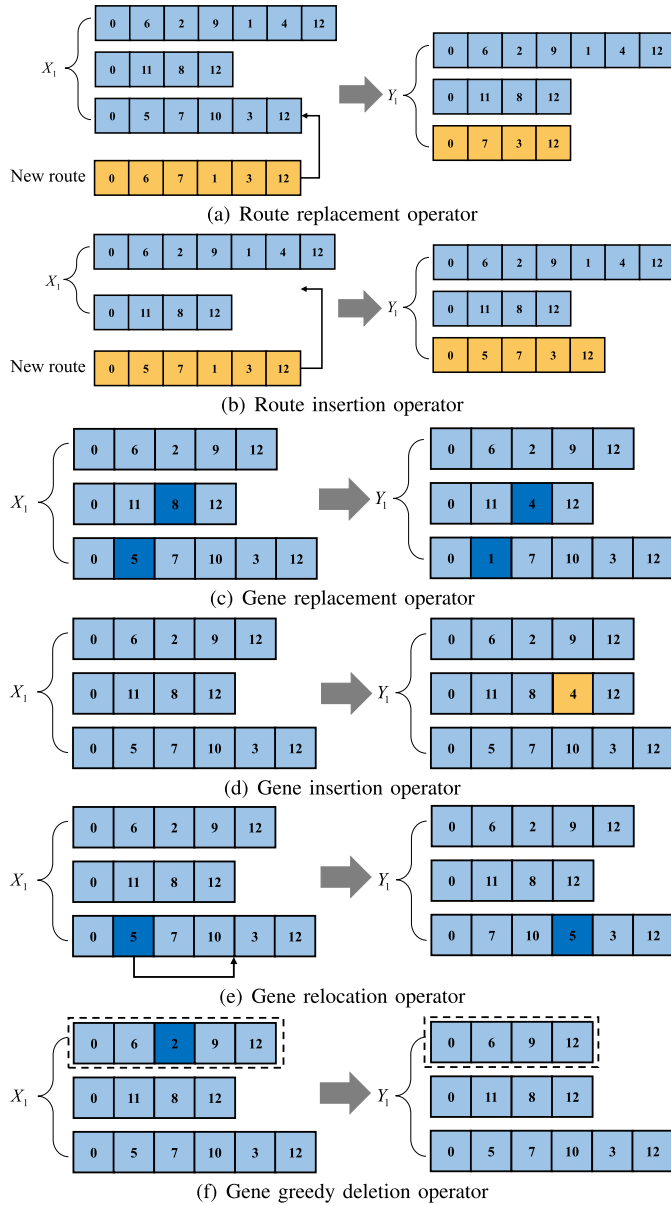


Fig. 8. Illustration of variable neighborhood search operators.

the value of p_m at $[0.2, 0.25, 0.3, 0.35, 0.4]$, and run the PGA-VNS ten times and record the average objective value for each combination of p_c and p_m . As shown in Table IV, the combination of $p_c = 0.8$ and $p_m = 0.25$ achieves the minimum. Therefore, we specify $p_c = 0.8$ and $p_m = 0.25$ in the following experiments.

Due to the fact that parallel computing will split the population into sub-populations, the instance ‘a40_5_6’ is selected to investigate the effect of different population sizes $[200, 400, 600, 800, 1000]$ on the PGA-VNS, and the results are shown in the Figure 9. As the population size increases, the optimal value of the PGA-VNS decreases slightly. This indicates that since sub-populations evolve independently, a larger population size would preserve the diversity of solutions and enhance the quality of parallel computing. In terms of CPU time, the computation time increases with the population size

TABLE IV
THE OBJECTIVE VALUES FOR DIFFERENT COMBINATIONS OF p_c AND p_m

p_c	p_m				
	0.2	0.25	0.3	0.35	0.4
0.5	2152.01	2116.43	2204.01	2160.44	2110.15
0.6	2108.77	2087.70	2114.67	2107.28	2181.25
0.7	2122.53	2031.34	2086.20	2086.49	2137.14
0.8	2000.03	1971.70	1990.91	2008.15	2020.57
0.9	2037.47	2020.50	2086.66	2080.82	2096.11

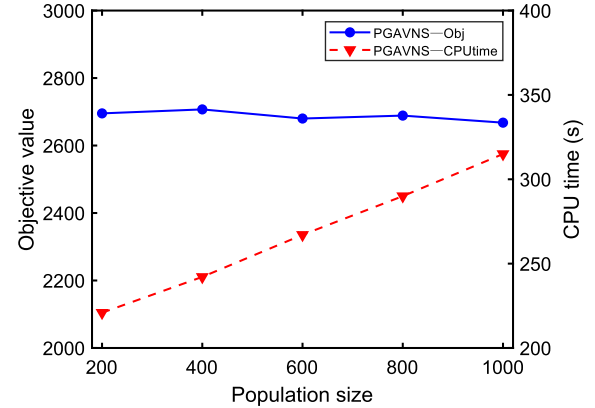


Fig. 9. Performance of PGA-VNS with different population sizes.

TABLE V
THE PARAMETER SETTING OF EACH ALGORITHM

Algorithm	Parameters
GA	$Iter=200, pop_size=400, p_c=0.8, p_m=0.25$
PSO ¹	$Iter=200, pop_size=400, c_1=0.3, c_2=0.3, w_s = 0.5, w_e = 0.2$
ALNS ²	$Iter=1000, S=6, r_1=40, r_2=30, r_3=20, r_4=10, \rho=0.4, d_T=0.9$
VNS	$Iter=200, S=6$
GA-ALNS	$Iter_GA=100, Iter_ALNS=900, pop_size=400, p_c=0.8, p_m=0.25, S=6, r_1=40, r_2=30, r_3=20, r_4=10, \rho=0.4, d_T=0.9$
GA-VNS	$Iter_GA=50, Iter_VNS=150, pop_size=400, p_c=0.8, p_m=0.25, S=6$
PGA-VNS	$Iter_GA=50, Iter_VNS=150, sub_pop_size=100, p_c=0.8, p_m=0.25, S=6$

¹ In PSO, the unique parameters c_1 and c_2 correspond to the cognitive learning factor and the social learning factor, respectively, while w_s and w_e represent the starting and ending values of the adaptive inertia weight [39]

² In ALNS, the unique parameter ρ represents the probability of the selection operator, with r_1, r_2, r_3 , and r_4 serving as the indices for the ρ updates, and d_T acting as the cooling factor in the Metropolis criterion [40]

amplifies. From a comprehensive perspective, we chose a population size of 400 in subsequent experiments.

B. Small and Medium-Scale Instances

This section conducts numerical experiments on 24 small and medium-scale instances with different numbers of fire spots, vehicle numbers, and capacities (detailed in Tables VI-VII). For each instance, the number of decision variables and the number of constraints are also provided in the columns *Var* and *Cons* respectively. To illustrate the effectiveness of the proposed PGA-VNS, we use two population-based algorithms GA and PSO, two single-solution-based search algorithms ALNS and VNS and two hybrid algorithms GA-ALNS and GA-VNS to solve each instance separately. The parameter settings of GA and GA-VNS are the same

as those of the PGA-VNS. The parameter setting of each algorithm is given in Table V. We use the solver Gurobi 9.1.1 with default settings to solve the MILP model. The computational time (CPU time) is constrained to 3600 s. Gurobi is used as the benchmark, and the relative error (Gap) of the objective value obtained by each algorithm is defined as follows

$$\text{Gap} = \frac{\text{Value of algorithm} - \text{Value of Gurobi}}{\text{Value of Gurobi}} \times 100\%. \quad (23)$$

Considering the randomness of the metaheuristic algorithm, we run each instance 10 times and record the average objective value (Loss), CPU computation time (CPU), and optimality gap for comparison (Gap). The last row of the table represents the mean values of all columns, and for negative values, we take the absolute value. The detailed comparison results are summarized in Tables VI-VII. Note that Gurobi could not solve instances in 3600 s with more than 15 fire spots, resulting in the best feasible solution found rather than the optimal solution. From a model-based perspective, We can observe that the number of variables and constraints increases sharply with the number of fire spots. However, this quantity remains consistent irrespective of the capacity of the vehicle. Furthermore, upon comparing the results of the instances in Table VI, we can observe that the objective value increases as the number of fire spots increases (comparing ‘a6_2_3’ and ‘a10_2_3’). Conversely, when the number of vehicles increases, the objective value decreases (comparing ‘a30_4_6’ and ‘a30_5_6’). On the other hand, from an algorithmic perspective, the average computation time of the PGA-VNS is 122.17, and the obtained Gap value is at most 22.33%. Comparing the two population-based search algorithms GA and PSO algorithm, it is shown that GA takes less CPU times and the optimality gap is slightly better than the PSO algorithm. Comparing the two single-solution-based search algorithms ALNS and VNS algorithms, the results show that the VNS algorithm could obtain higher quality solutions, which shows the advantage of the VNS algorithm in our problem. Furthermore, the hybrid algorithm can improve the search range of the problem and thus obtain a higher quality solution with more CPU times consumption. For example, in the instance ‘a20_3_6’, the GA-VNS reduces the loss by 3.71% compared to the GA, and the PGA-VNS further reduces the loss by 0.59% based on the GA-VNS with a time reduced by 19.71%. In summary, these results show that the PGA-VNS outperforms the other algorithms from a comprehensive perspective of time consumed and solution quality.

C. Large-Scale Instances

To further evaluate the performance of the PGA-VNS, the PGA-VNS and GA-VNS are selected to solve 16 larger-scale instances in which the number of fire spots varies from 40 to 70. The PGA-VNS and GA-VNS run 10 times separately for each instance, and the average objective value (Ave), minimum objective value (Min), and CPU computation

time (CPU) are recorded in Table VIII. For comparison, we define the percentage optimality gap for the GA-VNS compared to the PGA-VNS as follows

$$\text{Gap} = \frac{\text{Value of GA-VNS} - \text{Value of PGA-VNS}}{\text{Value of PGA-VNS}} \times 100\%. \quad (24)$$

The results show that the PGA-VNS finds better solutions than the GA-VNS over 15 out of 16 instances (comparing the minimum objective value found); the average objective value of the PGA-VNS is less than that of the GA-VNS over 12 out of 16 instances. This implies that the PGA-VNS has excellent exploitability and robustness to approximate the global optimal solution when solving large-scale problems. In addition, in terms of computation time, the PGA-VNS can save an average of 7.77% of CPU time compared to the GA-VNS, through the parallelized evolution process of multiple sub-populations.

D. Real-World Case

We introduced a real fire case in the northeast region of Portugal, with relevant data sourced from an open dataset.⁵ In this dataset, we selected data for 20 fire spots on a specific day, with coordinate positions randomly generated. The terrain slope data is referred to the data from Tian et al. [20], and the corresponding data can be found in the table IX. There are a total of 4 vehicles available with a maximum capacity of 4 each. The overall time range for the operations is set as [0, 240] with a time interval δ of 20, the η_i is supposed to be set as $\sqrt{R_i}$ and the value of B is set as 1000. Based on the description above, we tested the real case and presented detailed results obtained from four algorithms: GA, VNS, GA-VNS, and PGA-VNS. As shown in Figure 10, the square denotes the depot and the red circles represent fire spots. For each fire spot, the loss would increase with time, and the base loss is indicated by the blue number. The results obtained by different algorithms and the corresponding route diagram of the minimum solution as detailed in Table X. In the four algorithms, the hybrid algorithm GA-VNS produces better results than GA and VNS. The total loss is 759.57 and the time is 132.62. Moreover, the PGA-VNS introduces parallel computing based on GA-VNS, achieving the minimum solution of 741.19 with a shorter time of 104.93.

VI. EXTENSION: THE SCENARIO OF UAVS

In recent years, UAVs are quickly being applied in forest fire-fighting [46], because of their ability that easier fly into target-specific areas with a raging fire or limited visibility. Furthermore, they can gather data such as temperature, humidity and wind speed, allowing for more precise and informed decisions when attacking forest fires. In practice, UAVs can be sent to the fire spots and suppress the fire by dropping fire-fighting shells. The flight of UAVs is remotely controlled by

⁵<https://www.openml.org/search?type=data&sort=runs&id=42363&status=active>

TABLE VI
COMPARISON OF FOUR SINGLE ALGORITHMS ON SMALL-SCALE INSTANCES

Instance	Var	Cons	Gurobi		GA			PSO			ALNS			VNS		
			Loss	CPU(s)	Loss	CPU(s)	Gap	Loss	CPU(s)	Gap	Loss	CPU(s)	Gap	Loss	CPU(s)	Gap
a6_2_2	316	1604	1909.46	15.86	1909.46	28.65	0.00%	1909.46	41.27	0.00%	1909.46	80.37	0.00%	1909.46	74.67	0.00%
a6_2_3	316	1604	836.18	5.42	836.18	33.84	0.00%	836.18	43.66	0.00%	836.18	88.29	0.00%	836.18	78.92	0.00%
a6_3_2	474	2403	450.95	8.44	450.95	35.27	0.00%	450.95	59.67	0.00%	450.95	93.58	0.00%	450.95	77.38	0.00%
a8_2_2	440	2494	898.03	13.76	898.03	39.21	0.00%	898.03	58.41	0.00%	898.03	95.14	0.00%	898.03	82.12	0.00%
a8_2_3	440	2494	767.21	14.75	767.21	38.14	0.00%	767.21	60.19	0.00%	767.21	98.33	0.00%	767.21	88.89	0.00%
a8_3_2	660	3737	646.95	17.13	646.95	40.95	0.00%	646.95	68.32	0.00%	646.95	104.69	0.00%	646.95	94.20	0.00%
a10_2_3	580	3576	929.60	266.22	929.60	43.85	0.00%	929.60	72.45	0.00%	929.60	99.74	0.00%	929.60	90.71	0.00%
a10_3_2	870	5359	789.32	208.12	789.32	47.66	0.00%	789.32	77.84	0.00%	789.32	110.21	0.00%	789.32	93.74	0.00%
a10_3_3	870	5359	481.79	40.70	481.79	49.06	0.00%	481.79	74.26	0.00%	481.79	107.06	0.00%	481.79	105.87	0.00%
a15_3_3	1500	10674	1615.09	3600.00	1665.56	63.29	3.12%	1687.19	92.76	4.46%	1671.17	102.24	3.47%	1640.32	93.96	1.56%
a15_3_4	1500	10674	1401.67	3600.00	1218.71	59.73	-13.05%	1235.23	94.68	-11.87%	1220.28	96.86	-12.94%	1212.36	98.72	-13.51%
a15_4_3	2000	14227	1038.70	3600.00	1100.08	69.86	5.91%	1123.76	96.83	8.19%	1091.31	100.86	5.07%	1088.76	119.56	4.82%
a20_3_6	2280	17789	1809.00	3600.00	1081.37	76.52	-40.22%	1086.72	113.25	-39.93%	1064.80	112.71	-41.14%	1048.20	96.92	-42.06%
a20_4_5	3040	23712	1351.21	3600.00	1044.85	89.39	-22.67%	1082.09	109.47	-19.92%	1050.71	98.10	-22.24%	1031.34	95.38	-23.67%
a20_5_4	3800	29635	1099.62	3600.00	1096.77	92.14	0.26%	1106.53	110.23	0.63%	1079.91	107.77	-1.79%	1061.14	104.31	-3.50%
a25_4_5	4280	35597	2663.73	3600.00	1899.56	102.24	-28.69%	1916.85	125.51	-28.04%	1858.18	125.82	-30.24%	1797.11	100.74	-32.53%
a25_4_6	4280	35597	2174.44	3600.00	1232.87	117.61	-43.30%	1273.14	143.72	-41.45%	1232.87	114.93	-43.30%	1213.00	106.26	-44.22%
a25_5_4	5350	44490	1955.51	3600.00	1841.79	116.38	-5.82%	1859.66	152.03	-4.90%	1730.46	128.87	-11.51%	1711.34	112.54	-12.49%
a30_4_6	5720	49882	4175.90	3600.00	2668.52	127.48	-36.10%	2715.26	166.64	-34.98%	2615.33	133.69	-37.37%	2553.79	110.52	-38.84%
a30_4_7	5720	49882	4656.60	3600.00	1970.10	130.96	-57.69%	2005.56	174.30	-56.93%	1965.64	140.13	-57.79%	1924.77	122.05	-58.67%
a30_5_6	7150	62345	3313.18	3600.00	1852.33	141.52	-44.09%	1896.60	179.25	-42.76%	1840.39	148.25	-44.45%	1830.60	114.51	-44.75%
a35_5_6	9200	83200	4427.77	3600.00	1851.00	165.47	-58.20%	1908.93	203.49	-56.89%	1778.48	137.61	-59.83%	1775.71	117.38	-59.90%
a35_5_7	9200	83200	4039.06	3600.00	1323.20	158.32	-67.24%	1338.99	224.85	-66.85%	1276.11	140.36	-68.41%	1237.92	120.41	-69.35%
a35_6_5	11040	99833	6037.65	3600.00	1935.46	185.50	-67.94%	1989.63	233.43	-67.05%	1909.37	159.70	-68.38%	1869.68	129.34	-69.03%
Ave	3376	28307	2061.19	2274.60	1312.15	85.54	20.60%	1330.65	115.69	20.20%	1295.60	113.55	21.16%	1279.40	101.21	21.62%

TABLE VII
COMPARISON OF GA, GA-ALNS, GA-VNS AND PGA-VNS ON SMALL AND MEDIUM-SCALE INSTANCES

Instance	Var	Cons	Gurobi		GA			GA-ALNS			GA-VNS			PGA-VNS		
			Loss	CPU(s)	Loss	CPU(s)	Gap	Loss	CPU(s)	Gap	Loss	CPU(s)	Gap	Loss	CPU(s)	Gap
a6_2_2	316	1604	1909.46	15.86	1909.46	28.65	0.00%	1909.46	93.65	0.00%	1909.46	68.79	0.00%	1909.46	56.31	0.00%
a6_2_3	316	1604	836.18	5.42	836.18	33.84	0.00%	836.18	104.52	0.00%	836.18	77.62	0.00%	836.18	62.88	0.00%
a6_3_2	474	2403	450.95	8.44	450.95	35.27	0.00%	450.95	120.40	0.00%	450.95	91.74	0.00%	450.95	76.54	0.00%
a8_2_2	440	2494	898.03	13.76	898.03	39.21	0.00%	898.03	100.28	0.00%	898.03	68.13	0.00%	898.03	63.29	0.00%
a8_2_3	440	2494	767.21	14.75	767.21	38.14	0.00%	767.21	118.75	0.00%	767.21	82.25	0.00%	767.21	70.14	0.00%
a8_3_2	660	3737	646.95	17.13	646.95	40.95	0.00%	646.95	121.05	0.00%	646.95	96.93	0.00%	646.95	83.09	0.00%
a10_2_3	580	3576	929.60	266.22	929.60	43.85	0.00%	929.60	108.42	0.00%	929.60	105.38	0.00%	929.60	91.78	0.00%
a10_3_2	870	5359	789.32	208.12	789.32	47.66	0.00%	789.32	117.45	0.00%	789.32	116.97	0.00%	789.32	98.62	0.00%
a10_3_3	870	5359	481.79	40.70	481.79	49.06	0.00%	481.79	127.02	0.00%	481.79	119.48	0.00%	481.79	103.21	0.00%
a15_3_3	1500	10674	1615.09	3600.00	1665.56	63.29	3.12%	1615.09	108.41	0.00%	1615.09	113.60	0.00%	1615.09	96.76	0.00%
a15_3_4	1500	10674	1401.67	3600.00	1218.71	59.73	-13.05%	1213.44	115.59	-13.43%	1206.11	133.40	-13.95%	1207.67	113.40	-13.84%
a15_4_3	2000	14227	1038.70	3600.00	1100.08	69.86	5.91%	1090.40	108.16	4.98%	1097.00	128.84	5.61%	1084.15	106.28	4.38%
a20_3_6	2280	17789	1809.00	3600.00	1081.37	76.52	-40.22%	1047.67	122.15	-42.09%	1041.23	136.71	-42.44%	1035.13	109.76	-42.78%
a20_4_5	3040	23712	1351.21	3600.00	1044.85	89.39	-22.67%	1025.08	130.15	-24.14%	1016.17	147.26	-24.80%	1026.00	117.63	-24.07%
a20_5_4	3800	29635	1099.62	3600.00	1096.77	92.14	0.26%	1046.44	132.19	-4.84%	1036.72	156.26	-5.72%	1031.03	129.11	-6.24%
a25_4_5	4280	35597	2663.73	3600.00	1899.56	102.24	-28.69%	1781.76	136.48	-33.11%	1769.86	138.69	-33.56%	1770.25	105.28	-33.54%
a25_4_6	4280	35597	2174.44	3600.00	1232.87	117.61	-43.30%	1198.62	156.33	-44.88%	1163.72	153.38	-46.48%	1171.80	117.14	-46.11%
a25_5_4	5350	44490	1955.51	3600.00	1841.79	116.38	-5.82%	1704.81	179.20	-12.82%	1687.90	174.50	-13.69%	1686.30	150.02	-13.77%
a30_4_6	5720	49882	4175.90	3600.00	2668.52	127.48	-36.10%	2523.28	162.19	-39.58%	2499.45	181.91	-40.15%	2481.87	156.97	-40.57%
a30_4_7	5720	49882	4656.60	3600.00	1970.10	130.96	-57.69%	1817.67	182.38	-60.97%	1789.27	199.01	-61.58%	1742.64	163.84	-62.58%
a30_5_6	7150	62345	3313.18	3600.00	1852.33	141.52	-44.09%	1800.28	196.34	-45.66%	1748.30	237.12	-47.23%	1716.80	196.71	-48.18%
a35_5_6	9200	83200	4427.77	3600.00	1851.00	165.47	-58.20%	1768.61	195.94	-60.06%	1774.88	234.58	-59.91%	1775.71	206.35	-59.90%
a35_5_7	9200	83200	4039.06	3600.00	1323.20	158.32	-67.24%	1234.64	217.00	-69.43%	1212.33	254.47	-69.98%	1226.39	209.84	-69.64%
a35_6_5	11040	99833	6037.65	3600.00	1935.46	185.50	-67.94%	1820.01	232.68	-69.86%	1804.04	298.31	-70.12%	1792.57	247.19	-70.31%
Ave	3376	28307	2061.19	2274.60	1312.15	85.54	20.60%	1266.55	141.11	21.91%	1257.15	146.47	22.30%	1253.04	122.17	22.33%

a ground command center. Facing scenarios of using UAVs in forest fire-fighting, a routing problem with UAVs naturally arises for the ground command center.

To solve the routing problem with UAVs, we extend our basic model $P1$ for suitability. For the sake of convenience, β , D and p_i^k denote the load coefficient, the maximum flight

duration, and the number of fire-fighting shells loaded when UAV k leaves spot i , respectively. In the extended model, the objective function remains unchanged, while some constraints need to be modified. First, the capacity constraint (9) should be modified as the load capacity constraint for UAVs. Each UAV should not depart from the depot with a load that exceeds its

TABLE VIII
COMPARISON BETWEEN PGA-VNS AND GA-VNS ON LARGE-SCALE INSTANCES

Instance	PGA-VNS			GA-VNS					
	Ave	Min	CPU(s)	Ave	Gap	Min	Gap	CPU(s)	Gap
a40_5_6	2664.48	2325.16	242.13	2657.84	-0.25%	2379.16	2.32%	273.93	13.13%
a40_5_7	2636.94	2243.20	276.68	2746.66	4.16%	2470.08	10.11%	293.20	5.97%
a40_6_5	2264.21	1964.64	246.71	2228.63	-1.60%	2071.57	5.44%	274.40	11.22%
a40_6_6	2242.59	1885.51	289.78	2258.98	0.73%	1992.83	5.69%	313.15	8.07%
a50_5_8	5698.05	4881.15	386.25	5927.82	4.03%	5089.57	4.27%	411.40	6.51%
a50_5_9	5755.85	5105.24	415.50	5784.95	0.51%	5154.46	0.96%	446.33	7.42%
a50_6_7	4910.51	4389.57	434.96	4866.82	-0.90%	4758.44	8.40%	476.43	9.53%
a50_6_8	4666.23	4369.94	383.25	4775.41	2.34%	4448.18	1.79%	417.88	9.04%
a60-6-8	9187.73	8190.95	518.77	9167.70	-0.22%	8236.42	0.56%	534.26	2.99%
a60-6-9	8877.69	8153.40	692.60	8908.22	0.34%	8160.81	0.09%	713.33	2.99%
a60-7-7	7560.92	6722.49	556.10	7691.48	1.73%	6904.48	2.71%	588.70	5.86%
a60-7-8	7460.30	6594.79	823.54	7497.99	0.51%	7037.09	6.71%	869.71	5.61%
a70-7-8	9025.59	8088.77	498.74	9187.86	1.80%	8257.79	2.09%	585.75	17.45%
a70-7-9	9025.72	8344.35	524.85	9164.33	1.54%	8023.93	-3.99%	606.97	15.65%
a70-8-7	7910.55	7014.70	743.94	8197.17	3.62%	7165.69	2.15%	739.98	-0.54%
a70-8-8	7736.43	6588.47	1379.61	7934.79	2.56%	6910.01	4.88%	1426.63	3.41%
Ave	5611.62	4993.45	443.46	5669.26	1.47%	5150.84	3.93%	476.81	8.14%

TABLE IX
CASE RELATED DATA

Fire spot	λ_s^1	λ_ϕ	λ_ω^2	R_0^3
1	0.14	1.0	2.62	13.90
2	0.12	0.9	2.81	12.20
3	0.07	0.9	1.38	5.80
4	0.12	1.2	2.23	12.20
5	0.09	2.1	1.90	15.90
6	0.14	1.6	1.26	13.90
7	0.09	2.9	2.62	15.90
8	0.15	1.0	1.74	8.10
9	0.10	0.9	3.07	9.60
10	0.12	1.6	1.62	10.20
11	0.12	2.1	1.48	10.20
12	0.12	2.1	1.48	10.20
13	0.12	1.2	2.04	10.20
14	0.06	1.6	1.74	11.60
15	0.29	0.9	2.04	7.40
16	0.09	1.2	1.62	7.20
17	0.29	2.9	1.74	7.40
18	0.09	1.6	1.48	7.20
19	0.29	1.2	1.90	7.40
20	0.29	2.1	1.90	7.40

¹ Derived from the Duff Moisture Code (DMC) in the dataset

² Calculated from the dataset using the wind speed v_ω , where $\lambda_\omega = e^{0.1783v_\omega}$

³ Derived from the Initial Spread Index (ISI) in the dataset

capacity, that is

$$p_0^k \leq Q, \quad k = 1, 2, \dots, K. \quad (25)$$

The relationships between x_{ij}^k and p_i^k are depicted as follows

$$\begin{cases} p_j^k \leq p_i^k - \left\lfloor \frac{f_{i\delta} z_{i\delta}^k}{B} \right\rfloor + M \times (1 - x_{ij}^k) \\ p_j^k \geq p_i^k - \left\lceil \frac{f_{i\delta} z_{i\delta}^k}{B} \right\rceil - M \times (1 - x_{ij}^k), \end{cases} \quad (26)$$

where $k = 1, 2, \dots, K, i = 0, 1, 2, \dots, N, j = 1, 2, \dots, N, \delta = 1, 2, \dots, T$. In addition, the domain of variables p_i^k should be specified as non-negative:

$$p_i^k \geq 0, \quad k = 1, 2, \dots, K, i = 0, 1, 2, \dots, N. \quad (27)$$

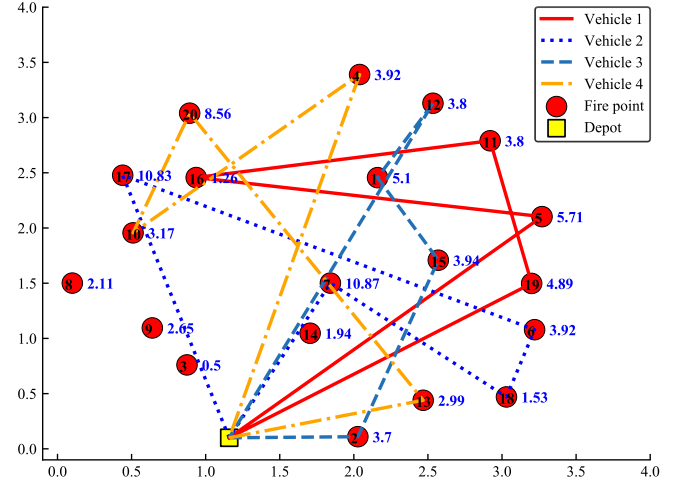


Fig. 10. The routes diagram of PGA-VNS.

Second, the flight time of UAVs is approximately linear with the payload [47], which arises an additional time captured by βp_i^k . Thus, the arrival times at the fire spots should satisfy the following constraints

$$\begin{cases} AT_j^k \leq AT_i^k + h \left\lceil \frac{f_{i\delta} z_{i\delta}^k}{B} \right\rceil + t_{ij} + \beta p_i^k + M(1 - x_{ij}^k) \\ AT_j^k \geq AT_i^k + h \left\lfloor \frac{f_{i\delta} z_{i\delta}^k}{B} \right\rfloor + t_{ij} + \beta p_i^k - M(1 - x_{ij}^k), \end{cases} \quad (28)$$

where $k = 1, 2, \dots, K, i = 0, 1, 2, \dots, N, j = 1, 2, \dots, N + 1, \delta = 1, 2, \dots, T$.

Third, the flight duration of UAVs is required less than the maximum flight duration D due to the limited capacity of the battery and the GPS communication signal limitation, that is

$$AT_{N+1}^k \leq D, \quad k = 1, 2, \dots, K. \quad (29)$$

TABLE X
THE RESULTS OBTAINED BY DIFFERENT ALGORITHMS

Algorithm	GA	VNS	GA-VNS	PGA-VNS
Vehicle1	0→12→6→3→7→21	0→6→10→20→4→21	0→17→12→16→7→21	0→19→11→16→5→21
Vehicle2	0→19→11→16→20→21	0→13→9→1→15→21	0→1→4→8→20→21	0→17→6→18→7→21
Vehicle3	0→2→15→10→5→21	0→17→12→8→7→21	0→2→15→10→6→21	0→2→15→1→12→21
Vehicle4	0→1→4→8→17→21	0→2→11→5→19→21	0→19→5→11→13→21	0→13→20→10→4→21
Loss	837.47	774.25	759.57	741.19
Time(s)	88.56	96.34	132.62	104.93

TABLE XI
TRAVEL DISTANCE BETWEEN THE DEPOT AND FIRE SPOTS (KM)

	0	1	2	3	4	5	6	7
0	0	42	56	63	65	50	66	45
1	42	0	20	33	45	35	48	64
2	56	20	0	46	53	44	42	58
3	63	33	46	0	60	55	52	54
4	65	45	53	60	0	62	64	56
5	50	35	44	55	62	0	65	66
6	66	48	42	52	64	65	0	63
7	45	64	58	54	56	66	63	0

Finally, the extension model $P2$ can be formulated as follows

$$\begin{cases} \min & C \\ \text{s.t.} & \text{Constraints (5)-(8), (10), (12)-(17), (25)-(29).} \end{cases} \quad (30)$$

We study a real-world case in Daxinganling, China in 2010 [48] to evaluate the extension model. In this case, 7 fire spots are involved and 3 UAVs with the capacity of 2 shells are available, while the location of the fire spots and the time-dependent loss are known. The travel distances between the depot and fire spots are shown in Table XI, and the bases of the fire losses are provided in Table XII. Assuming that the average speed of UAVs is 20 m/s, then the flight time between fire spots and the losses for each fire spot can be easily obtained. B is set at 20. Four scenarios with different fire-fighting operation time h , load coefficient β and maximum flight duration of UAVs D are considered for comparison, detailed in Table XIII.

The PGA-VNS is adopted to solve these 4 scenarios. Here the algorithm is slightly adjusted by introducing a decision variable p and adding a constraint on the maximum flight duration according to the extended model. The computational results are illustrated in Table XIII, including the total loss (Obj), the latest ending time (LET) and the routes of 3 UAVs. We can observe that the fire-fighting operation time h and the maximum flight duration of UAVs D have significant impacts on the routing decision. By comparing the results under scenario 4 and scenario 3, we can find that the total loss decreases by 43.77% when h decreases from 15 minutes to 5 minutes. Similarly, total loss decreases by 67.81% when D increases from 60 minutes (in scenario 2) to 100 minutes (in scenario 3). These results imply that a shorter fire-fighting operation time and a longer maximum endurance of UAVs can greatly reduce the total loss. In addition, UAVs with a lower load coefficient can visit more fire spots, resulting in a

TABLE XII
THE TIME-VARYING LOSSES OF 7 FIRE SPOTS

Fire spot	Loss						
	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$	$t=6$	$t=7$
1	2.27	4.84	7.55	11.72	17.27	24.51	33.82
2	1.48	2.06	2.64	3.26	3.93	4.65	5.43
3	1.59	2.35	3.17	4.07	5.08	6.20	7.45
4	2.64	5.91	10.96	18.44	29.17	44.17	64.67
5	2.56	5.58	10.15	16.80	26.19	39.13	56.61
6	2.19	4.22	6.96	10.62	15.40	21.55	29.35
7	1.84	3.06	4.52	6.27	8.37	10.87	13.82

TABLE XIII
COMPUTATIONAL RESULTS UNDER DIFFERENT SCENARIOS

Scenario	h	β	D	Obj	LET	Vehicle 1	Vehicle 2	Vehicle 3
1	5	0.4	100	18.54	99.50	0→7→4→8	0→5→3→8	0→1→6→8
2	5	0.8	60	63.52	60.00	0→4→8	0→1→8	0→5→8
3	5	0.8	100	20.45	98.17	0→7→4→8	0→5→2→8	0→1→6→8
4	15	0.8	100	36.37	98.17	0→4→8	0→1→5→8	0→6→8

lower loss. By comparing scenario 1 and scenario 3, we can see that when the load coefficient β increases from 0.4 to 0.8, the optimal routes change from 0→7→4→8, 0→5→3→8, 0→1→6→8 to 0→7→4→8, 0→5→2→8, 0→1→6→8. Due to the low-load UAVs visiting one different fire spots (i.e., fire spots 2 and 3), the drone can visit a more serious fire spot, then the total loss decreases by 9.34%.

To further investigate the impact of parameters (i.e., capacity, the load coefficient, flight duration, and operation time) on the total loss, we conduct sensitivity analysis on the instance 'a25_4_5'. Still using the parameter setting under scenario 3, we change one of the parameters while keeping the others unchanged. The results are shown in Figure 11. Figure 11(a) shows that as the number and capacity of UAVs increase, the rescue capability is enhanced, resulting in a dramatic reduction of the total loss. Figure 11(b) shows that a larger load coefficient β of UAVs would delay rescue efforts, thus leading to a higher loss. Analysis of the flight duration shows that escalating flight duration can effectively decrease the total loss, which is stated in Figure 11(c). Figure 11(d) illustrates the analysis result of fire-fighting operation time and suggests that shortening the fire-fighting operation time is an effective way to reduce the total loss. Hence, two types of countervailing forces determine the aggregate effect on the loss. An increase in capacity and flight duration is always beneficial for the decrease of the total loss, while the increase in the load coefficient and operation time offsets this benefit, leading to an increase of the total loss. Understanding the impact of these

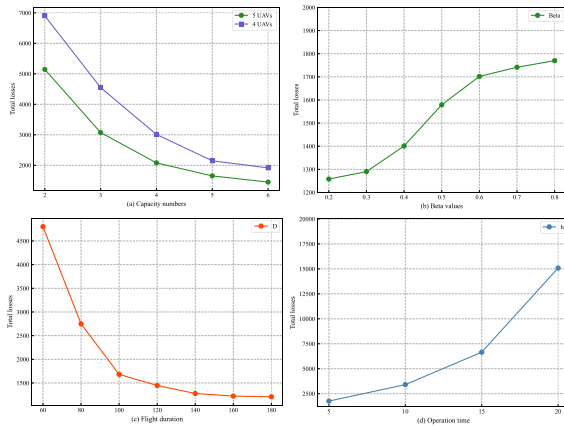


Fig. 11. Parameters sensitivity analysis.

two types of forces can help us improve fire-fighting efficiency and reduce the total loss.

VII. CONCLUSION

This study investigates the VRP for fire engines/UAVs in forest fire-fighting with the goal of minimizing the accumulated losses across all fire spots. An MILP model is formulated to optimize the routes of fire engines. A hybrid PGA-VNS metaheuristic has been developed for searching high quality solutions. The experimental results indicate that the PGA-VNS can obtain more satisfactory solutions than other baseline algorithms for various-scale instances. In particular, PGA-VNS exhibits an average loss reduction of 0.33% compared to GA-VNS, while also achieving an average decrease in computation time by 16.59% for small and medium-scale instances. Moreover, we extend the model to the scenario of UAVs. A real-world forest fire case and an instance with 25 fire spots validate the extended model and demonstrate the applicability of the proposed algorithm. The results of the sensitivity analysis show that increasing capacity and flight endurance is beneficial for reducing the total loss; on the contrary, an increase in load factor and operation time can lead to an increase of the total loss. This research could support decision makers in minimizing the total loss by determining the most efficient rescue routes for fire engines or UAVs in forest fire incidents.

Several interesting research directions following this study can be conducted in the future. First, this study paper proposes a routing method for a single type of fire-fighting equipment (i.e., fire engines or UAVs). Future research can be conducted on the coordinated routing problem for fire engines and UAVs, in which fire engines not only undertake the fire-fighting task but also serve as a platform for UAVs to replenish shells. In addition, new fire spots may continuously appear during the fire-fighting process, which requires a rolling-horizon optimization approach for the fire engines and/or UAVs.

REFERENCES

- [1] P. Wu, F. Chu, A. Che, and M. Zhou, "Bi-objective scheduling of fire engines for fighting forest fires: New optimization approaches," *IEEE Trans. Intell. Transp. Syst.*, vol. 19, no. 4, pp. 1140–1151, Apr. 2018.
- [2] G. Macrina, L. Di Puglia Pugliese, F. Guerriero, and G. Laporte, "Drone-aided routing: A literature review," *Transp. Res. Part C: Emerg. Technol.*, vol. 120, Nov. 2020, Art. no. 102762.

- [3] A. Reatti, A. Casazza, R. Fiorenzani, G. Mattei, L. Pugi, and A. Mela, "A fixed wing UAV with VTOL capabilities: Design, control and energy management," *Int. J. Model., Identificat. Control*, vol. 41, no. 3, p. 206, 2022.
- [4] W. Peng, W. Lubing, and C. Chengbin, "Forest fires emergency resource scheduling considering rescue priority under resource constraints," *J. Syst. Sci. Math. Sci.*, vol. 41, no. 12, pp. 3461–3477, Dec. 2021.
- [5] B. M. Baker and M. A. Ayechew, "A genetic algorithm for the vehicle routing problem," *Comput. Oper. Res.*, vol. 30, no. 5, pp. 787–800, Apr. 2003.
- [6] P. Qi, Y. Wang, and X. Lin, "Optimization of reverse logistics network for medical waste recycling," *J. Data, Inf. Manage.*, vol. 5, nos. 1–2, pp. 71–88, Apr. 2023.
- [7] M. E. H. Sadati and B. Çatay, "A hybrid variable neighborhood search approach for the multi-depot green vehicle routing problem," *Transp. Res. Part E, Logistics Transp. Rev.*, vol. 149, May 2021, Art. no. 102293.
- [8] X. Li, Z. Pan, H. Ma, and B. Zhang, "Neural network-based subway regenerative energy optimization with variable headway constraints," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 9, pp. 9698–9711, May 2023, doi: 10.1109/TITS.2023.3270932.
- [9] X. Wang, T.-M. Choi, H. Liu, and X. Yue, "Novel ant colony optimization methods for simplifying solution construction in vehicle routing problems," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 11, pp. 3132–3141, Nov. 2016.
- [10] X. Zheng, X. Li, Y. Li, and Y. Liu, "An improved artificial bee colony algorithm based on cat mapping and differential variation," *J. Data, Inf. Manage.*, vol. 4, no. 2, pp. 119–135, Jun. 2022.
- [11] X. Wang, S. Shao, and J. Tang, "Iterative local-search heuristic for weighted vehicle routing problem," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3444–3454, Jun. 2021.
- [12] I. Roozbeh, M. Ozlen, and J. W. Hearne, "An adaptive large neighbourhood search for asset protection during escaped wildfires," *Comput. Oper. Res.*, vol. 97, pp. 125–134, Sep. 2018.
- [13] D.-D. Zhu, J.-Q. Sun, and Y. Zhao, "A hybrid GA-SA for the urgent patients disturbed physical examination rescheduling problem considering setup time," *IEEE Access*, vol. 9, pp. 14787–14806, 2021.
- [14] M. Hamian, A. Darvishan, M. Hosseinzadeh, M. J. Lariche, N. Ghadimi, and A. Nouri, "A framework to expedite joint energy-reserve payment cost minimization using a custom-designed method based on mixed integer genetic algorithm," *Eng. Appl. Artif. Intell.*, vol. 72, pp. 203–212, Jun. 2018.
- [15] O. Dib, L. Moalic, M.-A. Manier, and A. Caminada, "An advanced GA-VNS combination for multicriteria route planning in public transit networks," *Expert Syst. Appl.*, vol. 72, pp. 67–82, Apr. 2017.
- [16] A. Amirteimoori, I. Mahdavi, M. Solimanpur, S. S. Ali, and E. B. Tirkolaee, "A parallel hybrid PSO-GA algorithm for the flexible flow-shop scheduling with transportation," *Comput. Ind. Eng.*, vol. 173, Nov. 2022, Art. no. 108672.
- [17] W. Yu, Y. Chen, and M. Guan, "Hierarchical siting of macro fire station and micro fire station," *Environ. Planning B: Urban Analytics City Sci.*, vol. 48, no. 7, pp. 1972–1988, Sep. 2021.
- [18] I. Sung and T. Lee, "Optimal allocation of emergency medical resources in a mass casualty incident: Patient prioritization by column generation," *Eur. J. Oper. Res.*, vol. 252, no. 2, pp. 623–634, Jul. 2016.
- [19] T. Senapati, G. Chen, R. Mesiar, R. R. Yager, and A. Saha, "Novel Aczel-Alsina operations-based hesitant fuzzy aggregation operators and their applications in cyclone disaster assessment," *Int. J. Gen. Syst.*, vol. 51, no. 5, pp. 511–546, Feb. 2022.
- [20] G. Tian, A. M. Fathollahi-Fard, Y. Ren, Z. Li, and X. Jiang, "Multi-objective scheduling of priority-based rescue vehicles to extinguish forest fires using a multi-objective discrete gravitational search algorithm," *Inf. Sci.*, vol. 608, pp. 578–596, Aug. 2022.
- [21] X. Li, X. Liu, H. Ma, and S. Hu, "Integrated routing optimization for post-disaster rapid-detailed need assessment," *Int. J. Gen. Syst.*, vol. 49, no. 5, pp. 521–545, Jul. 2020.
- [22] G. Tian, Y. Ren, and M. Zhou, "Dual-objective scheduling of rescue vehicles to distinguish forest fires via differential evolution and particle swarm optimization combined algorithm," *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 11, pp. 3009–3021, Nov. 2016.
- [23] G. B. Dantzig and J. H. Ramser, "The truck dispatching problem," *Manag. Sci.*, vol. 6, pp. 80–91, Oct. 1959.
- [24] Sh. Shahparvari, B. Abbasi, P. Chhetri, and A. Abareshi, "Vehicle routing and scheduling for bushfire emergency evacuation," in *Proc. IEEE Int. Conf. Ind. Eng. Eng. Manage. (IEEM)*, Dec. 2015, pp. 696–700.

- [25] S. Shahparvari, B. Abbasi, P. Chhetri, and A. Abareshi, "Fleet routing and scheduling in bushfire emergency evacuation: A regional case study of the black Saturday bushfires in Australia," *Transp. Res. D, Transp. Environ.*, vol. 67, pp. 703–722, Feb. 2019.
- [26] Y. Fang, Y. Deng, and Z. Zhou, "Formulation and solution of an emergency routing problem for forest fire with priority areas," in *Proc. Int. Conf. Ind. Eng. Syst. Manage. (IESM)*, Sep. 2019, pp. 1–5.
- [27] C. Yuan, Y. Zhang, and Z. Liu, "A survey on technologies for automatic forest fire monitoring, detection, and fighting using unmanned aerial vehicles and remote sensing techniques," *Can. J. Forest Res.*, vol. 45, no. 7, pp. 783–792, Jul. 2015.
- [28] M. Kumar and K. Cohen, "Wild land fire fighting using multiple uninhabited aerial vehicles," in *Proc. AIAA Infotech@Aerospace Conf.*, Apr. 2009, p. 1857.
- [29] X. Chen, Y. Hu, and G. Xu, "A study on the multiple UAVs cooperative fire fighting based on consensus algorithm," *Int. J. Control Autom.*, vol. 8, no. 8, pp. 309–324, Aug. 2015.
- [30] D. A. Saikin, T. Baca, M. Gurtner, and M. Saska, "Wildfire fighting by unmanned aerial system exploiting its time-varying mass," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 2674–2681, Apr. 2020.
- [31] H. S. Munawar, F. Ullah, S. I. Khan, Z. Qadir, and S. Qayyum, "UAV assisted spatiotemporal analysis and management of bushfires: A case study of the 2020 Victorian bushfires," *Fire*, vol. 4, no. 3, p. 40, Jul. 2021.
- [32] J. F. Ehmke, A. M. Campbell, and B. W. Thomas, "Vehicle routing to minimize time-dependent emissions in urban areas," *Eur. J. Oper. Res.*, vol. 251, no. 2, pp. 478–494, Jun. 2016.
- [33] O. Ozkan, "Optimization of the distance-constrained multi-based multi-UAV routing problem with simulated annealing and local search-based matheuristic to detect forest fires: The case of Turkey," *Appl. Soft Comput.*, vol. 113, Dec. 2021, Art. no. 108015.
- [34] A. M. Altabeeb, A. M. Mohsen, and A. Ghallab, "An improved hybrid firefly algorithm for capacitated vehicle routing problem," *Appl. Soft Comput.*, vol. 84, Nov. 2019, Art. no. 105728.
- [35] Y. Wu, F. Pan, S. Li, Z. Chen, and M. Dong, "Peer-induced fairness capacitated vehicle routing scheduling using a hybrid optimization ACO-VNS algorithm," *Soft Comput.*, vol. 24, no. 3, pp. 2201–2213, Feb. 2020.
- [36] J.-F. Cordeau and M. Maischberger, "A parallel iterated Tabu search heuristic for vehicle routing problems," *Comput. Oper. Res.*, vol. 39, no. 9, pp. 2033–2050, Sep. 2012.
- [37] V. Roberge, M. Tarbouchi, and G. Labonte, "Comparison of parallel genetic algorithm and particle swarm optimization for real-time UAV path planning," *IEEE Trans. Ind. Informat.*, vol. 9, no. 1, pp. 132–141, Feb. 2013.
- [38] X.-Y. Zhang, J. Zhang, Y.-J. Gong, Z.-H. Zhan, W.-N. Chen, and Y. Li, "Kuhn-Munkres parallel genetic algorithm for the set cover problem and its application to large-scale wireless sensor networks," *IEEE Trans. Evol. Comput.*, vol. 20, no. 5, pp. 695–710, Oct. 2016.
- [39] J. Xin, G. Chen, and Y. Hai, "A particle swarm optimizer with multi-stage linearly-decreasing inertia weight," in *Proc. Int. Joint Conf. Comput. Sci. Optim.*, vol. 1, Apr. 2009, pp. 505–508.
- [40] H. C. Lau and B. Li, "Solving the winner determination problem for online B2B transportation matching platforms," *Transp. Res. Part E, Logistics Transp. Rev.*, vol. 150, Jun. 2021, Art. no. 102324.
- [41] A. S. Tasan and M. Gen, "A genetic algorithm based approach to vehicle routing problem with simultaneous pick-up and deliveries," *Comput. Ind. Eng.*, vol. 62, no. 3, pp. 755–761, Apr. 2012.
- [42] P. Hansen, N. Mladenović, J. Brimberg, and J. A. M. Pérez, *Variable Neighborhood Search*. Springer, Oct. 2019.
- [43] H. Ma, M. Yang, and X. Li, "Integrated optimization of customized bus routes and timetables with consideration of holding control," *Comput. Ind. Eng.*, vol. 175, Jan. 2023, Art. no. 108886.
- [44] J. Kytöjoki, T. Nuortio, O. Bräysy, and M. Gendreau, "An efficient variable neighborhood search heuristic for very large scale vehicle routing problems," *Comput. Operations Res.*, vol. 34, no. 9, pp. 2743–2757, Sep. 2007.
- [45] R. Jain et al., "Application of proposed hybrid active genetic algorithm for optimization of traveling salesman problem," *Soft Comput.*, vol. 27, no. 8, pp. 4975–4985, Oct. 2022.
- [46] R. G. Ribeiro, J. R. C. Júnior, L. P. Cota, T. A. M. Euzébio, and F. G. Guimarães, "Unmanned aerial vehicle location routing problem with charging stations for belt conveyor inspection system in the mining industry," *IEEE Trans. Intell. Transport. Syst.*, vol. 21, no. 10, pp. 4186–4195, Oct. 2020.
- [47] K. Dorling, J. Heinrichs, G. G. Messier, and S. Magierowski, "Vehicle routing problems for drone delivery," *IEEE Trans. Syst. Man, Cybern., Syst.*, vol. 47, no. 1, pp. 70–85, Jan. 2017.
- [48] Z. Yang, L. Guo, and X. Dong, "Research on optimal allocation of rescue resources for forest fire in daxing-anling," *J. Dalian Univ. Tech.*, vol. 54, no. 6, pp. 644–650, Nov. 2014.



Xiang Li received the Ph.D. degree in operations research and control from Tsinghua University, Beijing, China, in 2008. He is currently a Professor with the School of Economics and Management Science, Chang'an University, China. He has authored two books and more than 130 articles on international journals, including *IEEE TRANSACTIONS ON FUZZY SYSTEMS*, *IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS*, *IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS*, *IEEE TRANSACTIONS ON ENGINEERING MANAGEMENT*, *European Journal of Operational Research*, *Transportation Research Part B: Methodological*, *Transportation Research Part C: Emerging Technologies*, *International Journal of Production Economics*, and *Information Sciences*. His research interests include intelligent transportation systems, optimization under uncertainty, and big data analysis. He serves as the Editor-in-Chief for *International Journal of General Systems* and *Journal of Data, Information and Management* and an Associate Editor for *Omega-International Journal of Management Science*, *Information Sciences*, and *Transportmetrica B: Transport Dynamics*.



Nan Chen received the master's degree in logistics engineering from Shijiazhuang Tiedao University in 2019. She is currently pursuing the Ph.D. degree in management science and engineering with Beijing University of Chemical Technology, China. Her current research interests include operations research, such as vehicle routing planning and scheduling, and big data analysis.



Hongguang Ma received the Ph.D. degree in control science and engineering from Beijing University of Chemical Technology, China, in 2019. He is currently a Lecturer with the School of Economics and Management, Beijing University of Chemical Technology. He has authored more than 30 articles in international journals, including *IEEE TRANSACTIONS ON FUZZY SYSTEMS*, *IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS*, *IEEE TRANSACTIONS ON ENGINEERING MANAGEMENT*, *Information Sciences*, and *Computers and Industrial Engineering*. His research interests include transportation management, operations research and optimization, and big data analysis. He has served as the Guest Editor for *Information Sciences*, *Soft Computing*, *Scientific Programming*, and *Axioms*.



Fapeng Nie is currently pursuing the Ph.D. degree in management science and engineering with Beijing University of Chemical Technology, China. His current research interests include the operation and management of smart transportation and supply chain management. He is the first author of journal articles in the above fields and has published the corresponding articles in international journals, such as *Omega*.



Xianzhe Wang received the degree in management science and engineering from the East China University of Science and Technology, Shanghai, China, in 2021. He is currently pursuing the degree in management science and engineering with Beijing University of Chemical Technology, Beijing, China. His current research interests include decision-making under model uncertainty, performance optimization and algorithm, and machine learning in operations research and operations management.